



武汉大学

WUHAN UNIVERSITY

第3讲 线性模型

《机器学习》

1 线性回归的基本模型

何为线性模型？

- **线性回归 (linear regression)** 是机器学习中预测数值型数据的最基础的方法。其目的是从样本中学习一个线性模型 $g(\mathbf{x})$ 去表征自变量（解释变量）和因变量（响应变量）之间的关系
- 简单线性回归：仅有一个自变量（样本特征）和一个响应变量（样本标记），二者之间的关系可以用一条直线近似表示
- 多元线性回归：有两个或两个以上的特征，且特征和标记之间是线性关系。当特征维数为2时，线性模型是一个平面。当特征维数3维及以上时，线性模型是一个超平面
- **多响应变量回归**：用于分析多个响应变量与特征之间的关系，可以看作是单响应变量回归的扩展

- 给定数据集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$, 样本空间由 d 维向量张成, $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{id})^T, y_i \in \mathbb{R}$, 则:

- 线性回归方程为:

$$\hat{y}_i = g(\mathbf{x}_i) = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id} + w_0$$

写为向量形式:

$$\hat{y}_i = g(\mathbf{x}_i) = \mathbf{x}_i^T \mathbf{w} + w_0,$$

其中 $\hat{y}_i$ 是回归预测值, $\mathbf{w} = (w_1, \dots, w_d)^T$ 和 w_0 是模型的参数



m 是样本个数, d 是样本维数。一定要区分开!

2 线性回归的学习准则

- 损失函数：定义在单样本上
- 代价函数：定义在全体样本上

➤ 一般采用MSE作为代价函数（Cost）。当回归预测值与样本标记值具有最小均方误差时获得最优参数

均方误差（Mean-Square Error, MSE）：

$$MSE = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

$$\begin{array}{ccc}
 \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \\ w_0 \end{bmatrix} & \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} & 1 \\ x_{21} & x_{22} & \cdots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{md} & 1 \end{bmatrix} & \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} \\
 \text{增广1维} \rightarrow & & \\
 \text{特征权重（参数）} & \text{样本特征属性值} & \text{样本标记值}
 \end{array}$$



➤ 最小化MSE，求解最优参数

- 最小化代价函数：

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

- 写成向量形式：

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

3 线性回归的优化算法

➤ 最小二乘法 (Ordinary least squares) 求解最优参数:

最小二乘法

$$\text{令 } L(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

← 平方误差代价函数

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = 2\mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}) = 0$$

← 求驻点

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

可求得:

$$\hat{y}_i = g(\mathbf{x}_i) = \mathbf{x}_i^T (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$



➤ 向量求导在机器学习中经常遇到

设 $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n)^T$, $\beta = (\beta_1, \beta_2, \dots, \beta_m)^T$, 则:

$$\frac{\partial \beta^T}{\partial \alpha} = \left(\frac{\partial \beta_1}{\partial \alpha}, \frac{\partial \beta_2}{\partial \alpha}, \dots, \frac{\partial \beta_m}{\partial \alpha} \right) = \begin{bmatrix} \frac{\partial \beta_1}{\partial \alpha_1} & \frac{\partial \beta_1}{\partial \alpha_2} & \dots & \frac{\partial \beta_1}{\partial \alpha_n} \\ \frac{\partial \beta_2}{\partial \alpha_1} & \frac{\partial \beta_2}{\partial \alpha_2} & \dots & \frac{\partial \beta_2}{\partial \alpha_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \beta_m}{\partial \alpha_1} & \frac{\partial \beta_m}{\partial \alpha_2} & \dots & \frac{\partial \beta_m}{\partial \alpha_n} \end{bmatrix}_{n \times m} \frac{\partial \beta}{\partial \alpha^T} = \left(\frac{\partial \beta_1}{\partial \alpha^T}, \frac{\partial \beta_2}{\partial \alpha^T}, \dots, \frac{\partial \beta_m}{\partial \alpha^T} \right)^T = \begin{bmatrix} \frac{\partial \beta_1}{\partial \alpha_1} & \frac{\partial \beta_1}{\partial \alpha_2} & \dots & \frac{\partial \beta_1}{\partial \alpha_n} \\ \frac{\partial \beta_2}{\partial \alpha_1} & \frac{\partial \beta_2}{\partial \alpha_2} & \dots & \frac{\partial \beta_2}{\partial \alpha_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \beta_m}{\partial \alpha_1} & \frac{\partial \beta_m}{\partial \alpha_2} & \dots & \frac{\partial \beta_m}{\partial \alpha_n} \end{bmatrix}_{m \times n}$$

$$\text{即: } \frac{\partial \beta^T}{\partial \alpha} = \left(\frac{\partial \beta}{\partial \alpha^T} \right)^T$$

一般行向量对列向量求导, 列向量对行向量求导

但很多时候为了简单起见, 会直接写为 $\frac{\partial \beta}{\partial \alpha}$



➤ 设 $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n)^T$, $f(\alpha) = (\beta_1, \beta_2, \dots, \beta_m)^T$, $g(\alpha) = (\gamma_1, \gamma_2, \dots, \gamma_m)^T$

$$h(f(\alpha)) = (\varepsilon_1, \varepsilon_2, \dots, \varepsilon_k)^T$$

则向量求导的基本法则:

$$\frac{\partial(\lambda f(\alpha) + \mu g(\alpha))^T}{\partial \alpha} = \lambda \frac{\partial f(\alpha)^T}{\partial \alpha} + \mu \frac{\partial g(\alpha)^T}{\partial \alpha}$$

$$\frac{\partial f(\alpha)^T g(\alpha)}{\partial \alpha} = \frac{\partial f(\alpha)^T}{\partial \alpha} g(\alpha) + \frac{\partial g(\alpha)^T}{\partial \alpha} f(\alpha)$$

$$\frac{\partial h(f(\alpha))^T}{\partial \alpha} = \frac{\partial f(\alpha)^T}{\partial \alpha} \cdot \frac{\partial h(f(\alpha))^T}{\partial f(\alpha)}$$

 $n \times k$ $n \times m$ $m \times k$

(向量求导链式法则)

➤ 设 $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n)^T$, A 为 $n \times m$ 矩阵, 则:

$$\frac{\partial \alpha^T A}{\partial \alpha} = A, \quad \frac{\partial \alpha^T A \alpha}{\partial \alpha} = (A + A^T) \alpha, \quad \frac{\partial \alpha^T \alpha}{\partial \alpha} = 2\alpha$$

➤ 所以在最小二乘中:

$$\begin{aligned} \frac{\partial (y - Xw)^T (y - Xw)}{\partial w} &= \frac{\partial (y - Xw)^T}{\partial w} \cdot \frac{\partial (y - Xw)^T (y - Xw)}{\partial (y - Xw)} \\ &= -X^T \cdot 2(y - Xw) = 2X^T (Xw - y) \end{aligned}$$

证明 $\frac{\partial \alpha^T A}{\partial \alpha} = A$ 留作课后思考题



➤ 若 $X^T X$ 为满秩矩阵, 则 $(X^T X)^{-1}$ 可求, 最优参数 w^* 可直接解出:

$$w^* = (X^T X)^{-1} X^T y$$

闭式解 closed-form



在机器学习中, 往往会遇到样本维数远大于样本个数 ($d > m$) 的情况, 比如图像或某种高分子样本等, $X^T X$ **不满秩**。

- 如果 $d > m$, 则 X 是一个行数小于列数的矩阵 (线性方程组的个数少于未知数的个数), 则 X 必不满秩
- 根据矩阵秩的性质: $\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$

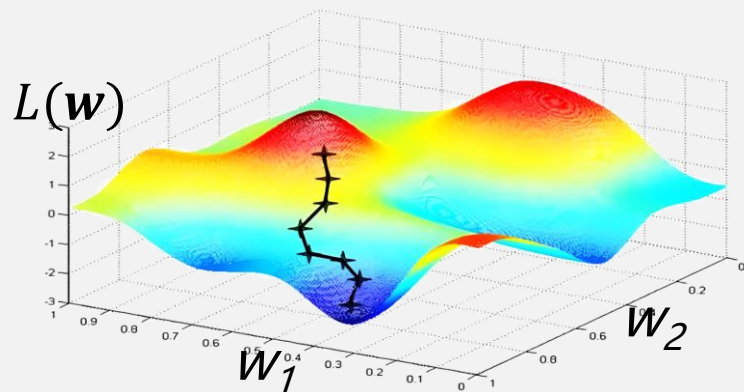
$X^T X$ 是一个方阵, 但不满秩

- 若 $X^T X$ 不可逆，可采用**梯度下降法**（Gradient descent）训练参数：

梯度下降法

- 确定代价函数： $L(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$ 为了求导方便
- 计算当前参数下代价函数的梯度。梯度是一个向量，其各分量表示代价函数对各个参数 w 的偏导数。梯度的方向代表函数值增加最快的方向，因此**负梯度方向**就是代价函数减小最快的方向
- 确定学习率 η （步长），根据步长沿着代价函数负梯度的方向迭代搜索，找到一个函数极值

$$\mathbf{w}^{(i+1)} \leftarrow \mathbf{w}^{(i)} - \eta \cdot \frac{\partial L(\mathbf{w}^{(i)})}{\partial \mathbf{w}^{(i)}}$$





$$\mathbf{w}^{(i+1)} \leftarrow \mathbf{w}^{(i)} - \eta \cdot \frac{\partial L(\mathbf{w}^{(i)})}{\partial \mathbf{w}^{(i)}}$$

← 上标(i)表示第 i 次迭代，每一次迭代都会更新一组参数 $\mathbf{w}$ ，从而得到一个新的代价（损失） $L(\mathbf{w})$

$$\nabla L(\mathbf{w}) = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \vdots \\ \frac{\partial L}{\partial w_d} \\ \frac{\partial L}{\partial w_0} \end{pmatrix} :$$

η :

梯度：是一个 $d+1$ 维向量(含对偏置参数 w_0 的偏导)，是函数变化最快的方向（负梯度：比喻为下山最快的方向）

学习率：每次更新参数的幅度。比喻为下山的步长。注意，学习率不一定是常数，是可以根据梯度下降情况而**动态**变化的



4 线性回归的正则化

- 岭回归 (Ridge regression) 和套索回归 (Lasso regression) 是带有**正则化 (regularization)** 的线性回归
- 正则化是通过对代价函数加以限制, **防止模型过拟合的重要方法**, 是结构风险最小化思想的一种具体体现, 在机器学习中非常有用。
- 在线性回归中一般会对参数和 (体现为**L1**范数或**L2**范数) 加以限制, 使得最优化时既让代价函数最小, 又满足参数限制条件。在多项式回归中, 则一般会对高次项的参数加以限制



- 岭回归在线性回归代价函数上添加了L2正则化项：

$$\mathbf{L}(\mathbf{w}) = \frac{1}{2m} \left[\sum_{i=1}^m (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^d w_j^2 \right] \quad (w_0 \text{不惩罚})$$

正则化系数 $\mathbf{w}$ 的L2范数的平方: $\|\mathbf{w}\|_2^2$

- 岭回归代价函数最小二乘优化的闭式解：

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$$

$$\mathbf{I} = \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{bmatrix}$$

是单位矩阵，如果保证 $(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})$ 可逆，则 $\mathbf{w}$ 具有闭式解（**也可从数据中心化角度解释**）



- 岭回归代价函数的梯度下降迭代:

对第 k 个特征的参数 w_k :

$$w_k^{(i+1)} \leftarrow w_k^{(i)} - \eta \cdot \left(\frac{\partial L(\mathbf{w})}{\partial w_k^{(i)}} + \frac{\lambda}{m} w_k^{(i)} \right)$$

- 岭回归正则化系数 λ 是一个对参数 $\mathbf{w}$ 的惩罚系数:

- 从 $L(\mathbf{w})$ 的角度看: 若 λ 系数很大, 则要使得 $L(\mathbf{w})$ 最小, 必须减小 $\mathbf{w}$, 即惩罚 $\mathbf{w}$, 降低各特征的权重系数。 λ 过大会使得 $\mathbf{w} \rightarrow [0, 0, \dots, 0]$, 模型可能会欠拟合
- 通过引入 λ 来限制了所有 w_k 之和, 惩罚降低不重要的那些特征的权重, 从而实现模型的结构风险最小化

提问: 既然岭回归有闭式解, 为什么还要用梯度下降法求参数?



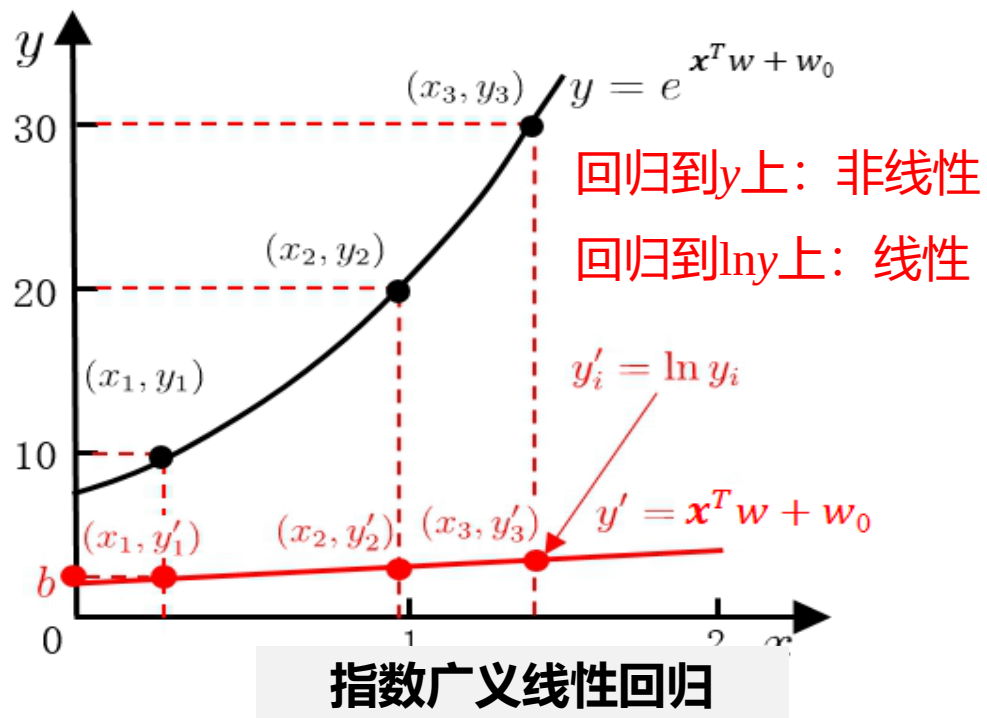
5 广义线性回归

- 广义线性回归 (Generalized Linear Regression, GLM) 是传统的线性回归的一种普遍拓展。其核心思想是通过某个**联系函数**，将线性回归的输出值映射成不同的分布
- 广义线性回归常用来处理标记空间 Y 具有**指数族分布** (exponential dispersion family) 的非线性回归问题。正态分布、泊松分布、二项分布、伽马分布、贝塔分布都属于指数族分布，其概率密度可以写成含有参数的指数形式

- 假设样本所对应的输出标记是在指数尺度上变化，可将输出标记的对数作为线性模型，构建回归方程

令 $y' = \ln y = \mathbf{x}^T \mathbf{w} + w_0$,实际是在用 $e^{\mathbf{x}^T \mathbf{w} + w_0}$ 逼近 y

- 在学习样本空间到标记空间的非线性函数映射过程中，**对数函数**起到了将线性回归的预测值与非线性的真实标记联系起来的作用





➤ 因此可以得到广义线性回归的基本形式：

$$\hat{y}_i = \sigma(z_i) = \sigma(\mathbf{x}_i^T \mathbf{w} + w_0)$$

其中， $\sigma()$ 是单调可微的**联系函数**（link function），也称**激活函数**



6 多项式回归

- 多项式回归 (Polynomial Regression) 是一种线性回归模型的推广。其中自变量 (解释变量) 和因变量 (响应变量) 之间的关系被建模为 n 次多项式
- 给定数据集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$, 样本空间由 d 维向量张成, $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{id})^T, y_i \in \mathbb{R}$, 则:

多项式回归方程为:

$$\hat{y}_i = g(\mathbf{x}_i) = w_0 + w_{11} x_{i1} + w_{12} x_{i2} + \dots + w_{1d} x_{id} + w_{21} x_{i1}^2 + \dots + w_n x_{i1} x_{i2} + \dots$$

偏置参数

一次项

高次项

交互项



➤ 对于非线性回归，增加高次项可以提高模型的拟合能力。但也要注意“**过拟合**”问题

- 高等数学的角度：任何**复杂**的函数 $f(x)$ 在点 $x=a$ 处具有任意阶导数，那么该函数可以在点 $x=a$ 处展开为如下形式的无穷级数

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \cdots + \frac{f^n(a)}{n!}(x-a)^n + \cdots$$

- 机器学习的角度：增加了高次项，就是增加了样本特征，从而提高了模型复杂度，提高模式识别能力

- 多项式回归可以等价为线性回归进行计算：通过一组**基函数** $\phi(x)$ 将 d 维（低维）的样本，映射到 n 维（高维）空间中，使得低维多项式回归等价于高维线性回归

$$g(x) = [\phi_i(x)]^T \mathbf{w} + w_0 \quad i=1, 2, \dots, n$$



• 幂函数基函数

比如为一维样本空间的样本 x_i 构造3个基函数： $x_i \xrightarrow{\phi(x)} \mathbf{z}_i = \begin{pmatrix} z_{i1} \\ z_{i2} \\ z_{i3} \end{pmatrix}$

$$z_{i1} = \phi_1(x_i) = x_i, \quad z_{i2} = \phi_2(x_i) = x_i^2, \quad z_{i3} = \phi_3(x_i) = x_i^3$$

则： $y_i = w_1 x_i + w_2 x_i^2 + w_3 x_i^3$

← 一元多项式回归

$$\Rightarrow y'_i = w_1 z_{i1} + w_2 z_{i2} + w_3 z_{i3}$$

← 多元线性回归

- 乘积项 (product term) 基函数

比如样本空间 $X=(x^{(1)}, x^{(2)})$ 是一个二维空间, 构造5个基函数:

$$z_{i1} = \phi_1(x_i) = x_{i1}, \quad \phi_2(x_i) = x_{i2}, \quad \phi_3(x_i) = x_{i1}^2, \quad \phi_4(x_i) = x_{i2}^2, \quad \phi_5(x_i) = x_{i1}x_{i2}$$

则: $y_i = w_1x_{i1} + w_2x_{i2} + w_3x_{i1}^2 + w_4x_{i2}^2 + w_5x_{i1}x_{i2}$ ← 二元多项式回归

$\Rightarrow y'_i = w_1z_{i1} + w_2z_{i2} + w_3z_{i3} + w_4z_{i4} + w_5z_{i5}$ ← 多元线性回归

- 基函数 $\phi(x)$ 方法降低了回归模型复杂度, 是一种将线性模型推广到非线性模型的代表性方法。但这种方法升高了样本空间的维数, 且需要知道基函数 $\phi(x)$ 的具体形式, 需要引入“核方法”对其进行进一步完善

9

模型小结

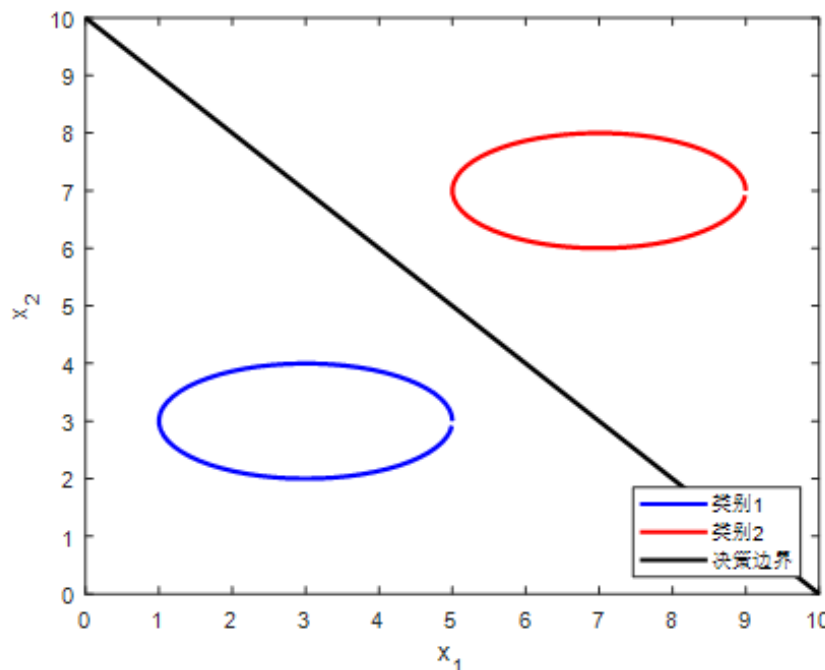
模型	策略（准则）	算法
$y = g(\mathbf{x}) = \mathbf{x}^T \mathbf{w} + w_0$ • 核心参数： 特征权向量 $\mathbf{w}$	• 最小均方误差MSE： $\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$	• 最小二乘法： $\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ • 梯度下降法： $\mathbf{w}^{(i+1)} \leftarrow \mathbf{w}^{(i)} - \eta \cdot \frac{\partial L(\mathbf{w}^{(i)})}{\partial \mathbf{w}^{(i)}}$

线性可分概念与线性分类算法

一个分类问题是否属于**线性可分**，取决于是否有可能**找到**一个点、直线、平面或超平面来分离两个相邻的类别。

如果每个类别样本的分布范围本身是全连通的单一凸集，且互不重叠，则这两个类别一定是线性可分的，如图所示。

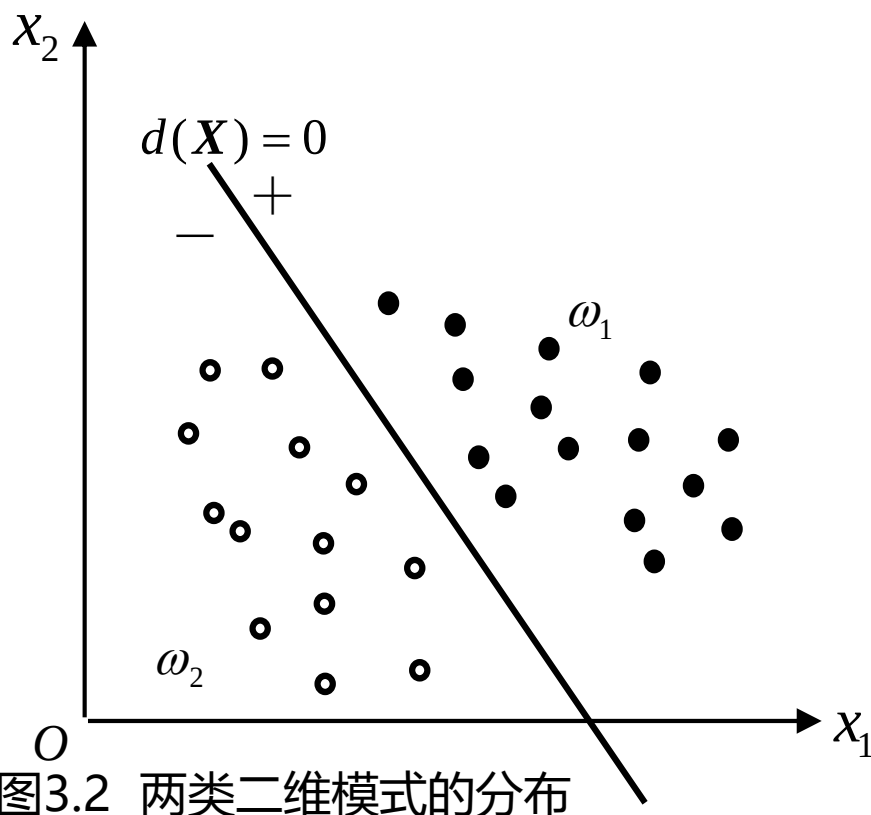
线性分类算法主要有线性判别函数、Fisher判别分析、单层感知器、逻辑回归等。



线性可分情况

直接用来对模式进行分类的**决策函数**。

若分属于 ω_1 , ω_2 两类的 n 维模式在空间中的分布区域, 可以用一代数方程 $d(X) = 0$ 决定的超平面作为分隔面, 两类样本分布在分隔面的两侧, 那么就称 $d(X)$ 为**判别函数**, 或称**决策函数**。代数方程 $d(X) = 0$ 表示的是 n 维空间的 $(n-1)$ 维判决面 (或超平面或超曲面, 视 $d(x)$ 形式而定)。

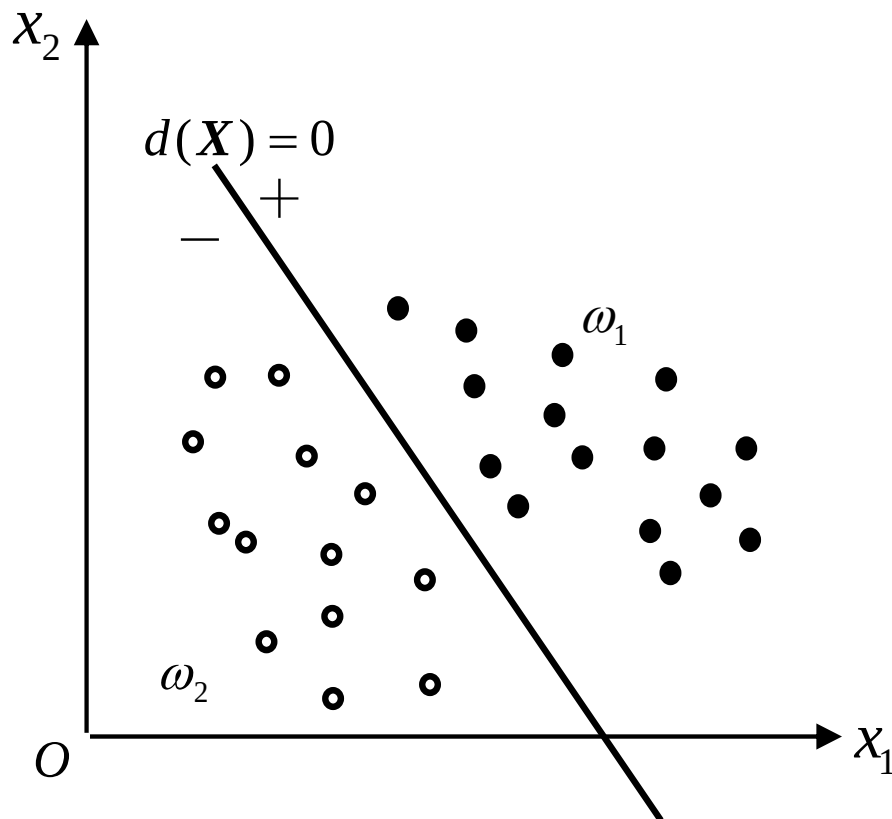


例: 一个二维的两类判别问题, 模式分布如图示, 这些分属于 ω_1 , ω_2 两类的模式可用平面中的直线 $d(X)=0$ 来划分。

$$d(X) = w_1x_1 + w_2x_2 + w_3 = 0$$

式中: x_1, x_2 为坐标变量,

w_1, w_2, w_3 为方程参数。



示例：线性判别函数（维数为2）。

将某一未知模式 X 代入：

$$d(X) = w_1x_1 + w_2x_2 + w_3$$

若 $d(X) > 0$ ，则 $X \in \omega_1$ 类；

若 $d(X) < 0$ ，则 $X \in \omega_2$ 类；

若 $d(X) = 0$ ，则 $X \in \omega_1$ 或 $X \in \omega_2$

或拒绝分类

特征维数=3时：判别面为三维空间中的**平面**。

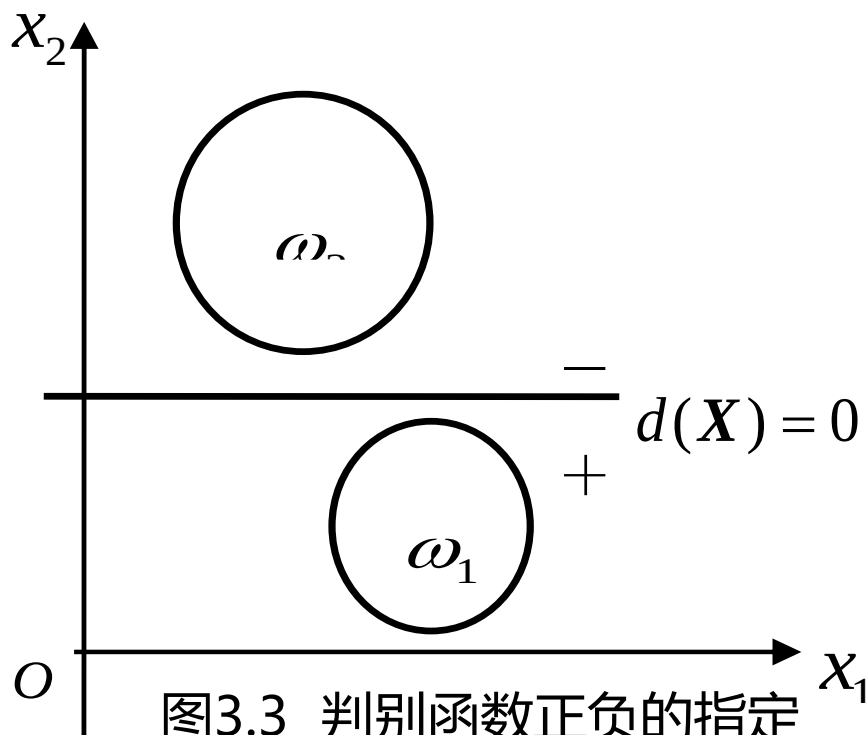
特征维数>3时：判别面为高维空间中的**超平面**。

注：为了清晰地了解 $d(x)$ 的含义，可在自变量（模式）空间中画出 $d(x)$ 取正负值的区域——这就是所谓**判别面的正侧、负侧**。

判别面的正负侧，是在**训练判别函数的权值时人为设定的**。但是一旦设定后，训练好的模型（分隔面）的正负侧就是确定的，而且由判别函数的梯度矢量确定。

一般，令第1类样本的函数值大于零，第2类样本的函数值小于零。也就是说，类别标号值为{1, -1}。也有用{1,0}表示正负类的。

具体值的大小不重要，主要便于算法公式的简洁。



(一) 线性判别函数的一般形式

将二维模式推广到 n 维, 线性判别函数的一般形式为:

$$d(X) = w_1x_1 + w_2x_2 + \dots + w_nx_n + w_{n+1} = \mathbf{W}_0^T \mathbf{X} + w_{n+1} \quad (3-2)$$

式中: $\mathbf{X} = [x_1, x_2, \dots, x_n]^T$

$\mathbf{W}_0 = [w_1, w_2, \dots, w_n]^T$: 权向量, 即参数向量。

增广向量形式:

$$d(X) = w_1x_1 + w_2x_2 + \dots + w_nx_n + w_{n+1} \cdot 1$$

$$= \begin{bmatrix} w_1 & w_2 & \dots & w_n & w_{n+1} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \\ 1 \end{bmatrix} = \mathbf{W}^T \mathbf{X}$$

式中:

$$\mathbf{X} = [x_1, x_2, \dots, x_n, 1]^T$$

为增广模式向量。

$$\mathbf{W} = [w_1, w_2, \dots, w_n, w_{n+1}]^T$$

为增广权向量,



1. 两类情况

$$d(\mathbf{X}) = \mathbf{W}^T \mathbf{X} \begin{cases} > 0, & \text{若 } \mathbf{X} \in \omega_1 \\ < 0, & \text{若 } \mathbf{X} \in \omega_2 \end{cases}$$

$d(\mathbf{X}) = 0$: 不可判别情况, 可随机地令 $\mathbf{X} \in \omega_1$ 或 $\mathbf{X} \in \omega_2$ 或拒绝分类。

2. 多类情况

对 M 个线性可分模式类, $\omega_1, \omega_2, \dots, \omega_M$, 有三种分类方式:

$\omega_i / \overline{\omega_i}$ 是非两分法

ω_i / ω_j 成对两分法

ω_i / ω_j 成对两分法 (无IR区)

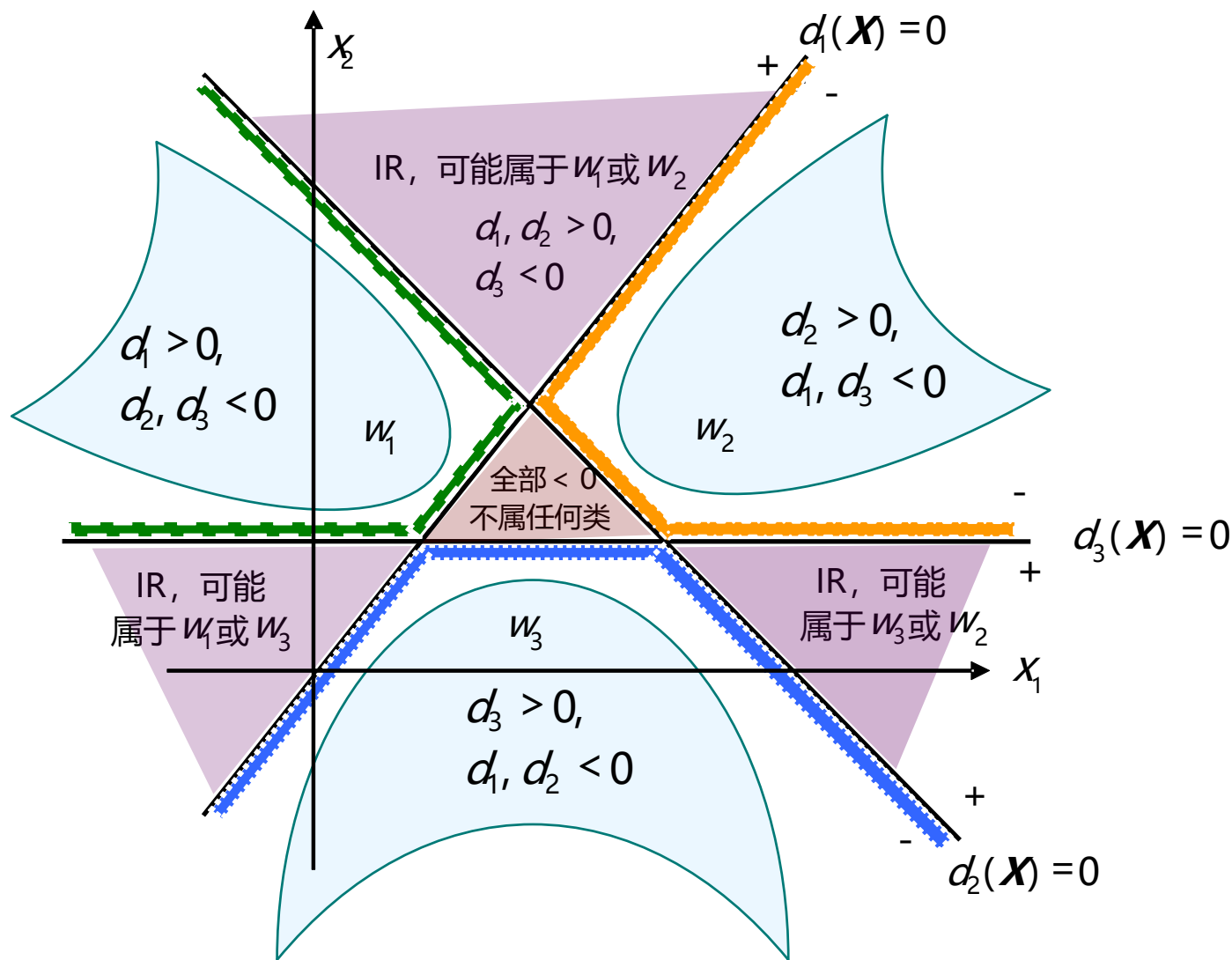
(1) 多类情况1: $\omega_i / \bar{\omega}_i$ 是非两分法

用线性判别函数将属于 ω_i 类的模式与其余不属于 ω_i 类的模式分开。能用本方法分类的模式集称为**整体线性可分的**。

$$d_i(\mathbf{X}) = \mathbf{W}_i^T \mathbf{X} \begin{cases} > 0, & \text{若 } \mathbf{X} \in \omega_i \\ < 0, & \text{若 } \mathbf{X} \in \bar{\omega}_i \end{cases} \quad i = 1, \dots, M$$

识别分类时:

将某个待分类模式 $\mathbf{X}$ 分别代入 M 个类的 $d(\mathbf{X})$ 中, 若只有 $d_i(\mathbf{X}) > 0$, 其他 $d_j(\mathbf{X})$ 均 < 0 , 则判为 ω_i 类。



对某一模式区, $d_i(X) > 0$ 的条件超过一个, 或全部的 $d_i(X) < 0$, 分类失效。相当于**不确定区** (indefinite region, **IR**)。

这一分法将 M 类问题转换成 M 个两类问题。在对一个样本进行分类时, 需要计算 M 个判别函数。



例 1 设有一个**二维三类**问题，其三个判别函数为：

$$d_1(X) = -x_1 + x_2 + 1 \quad d_2(X) = x_1 + x_2 - 4 \quad d_3(X) = -x_2 + 1$$

现有一模式， $\mathbf{X}=[7,5]^T$ ，试判定应属于哪类？并画出三类模式的分布区域。

解：将 $\mathbf{X}=[7,5]^T$ 代入上三式，有：

$$d_1(X) = -7 + 5 + 1 = -1 < 0$$

$$d_2(X) = 7 + 5 - 4 = 8 > 0$$

$$d_3(X) = -5 + 1 = -4 < 0$$

$$\because d_2(X) > 0, \quad d_1(X), d_3(X) < 0 \quad \therefore \mathbf{X} \in \omega_2$$

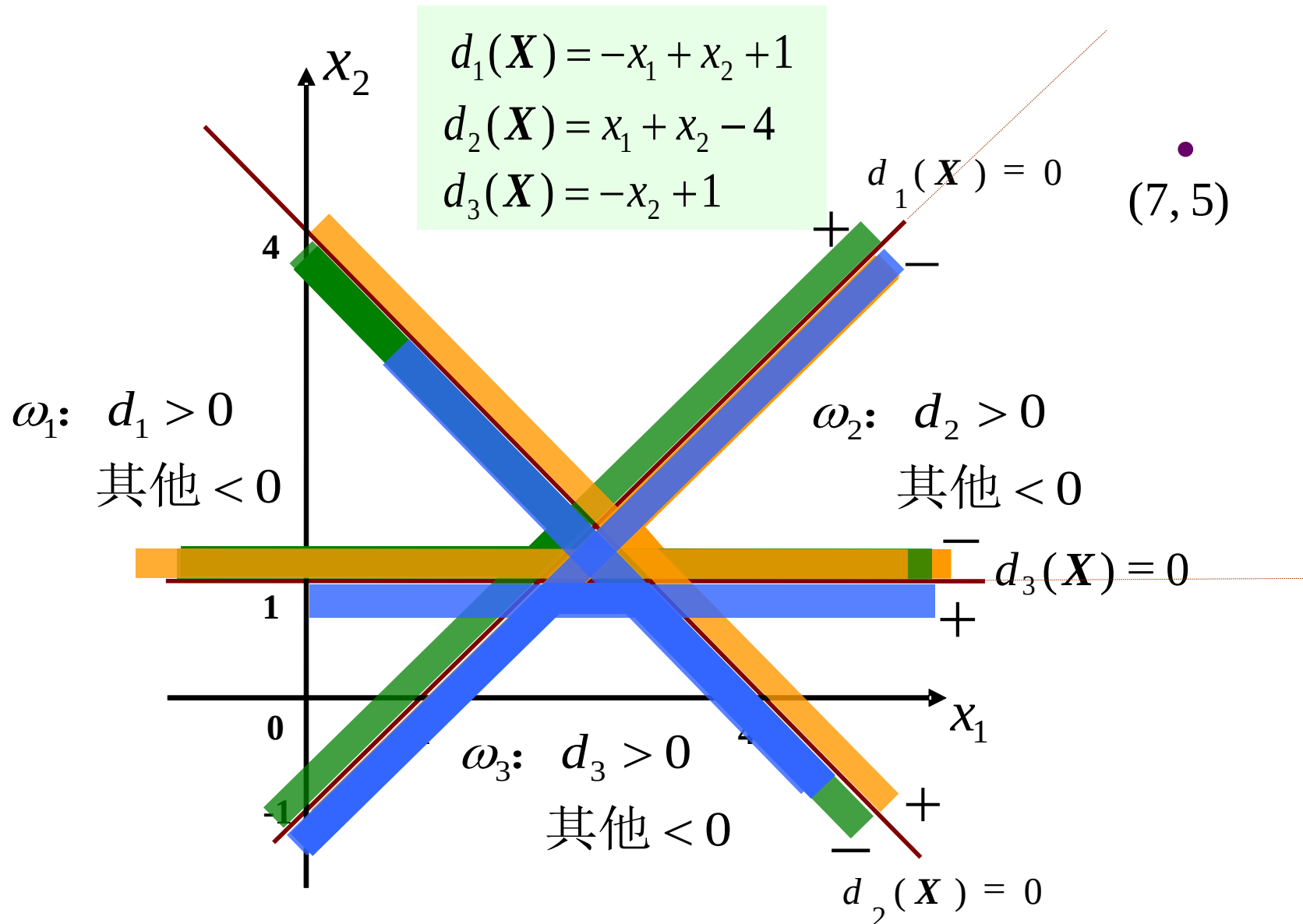
三个判别界面分别为：

$$-x_1 + x_2 + 1 = 0$$

$$x_1 + x_2 - 4 = 0$$

$$-x_2 + 1 = 0$$

图示如下：



步骤:

a) 画出界面直线。

b) 判别界面正负侧: 找特殊点代入。或者: 画矢量 W_0 (以原点为起点), 箭头指向的方向就是 $d(X)$ 的正侧

c) 找交集 (一正测、多负则侧)。

(2) 多类情况2: ω_i/ω_j 成对两分法

一个判别界面只能分开两个类别，不需要把其余所有的类别都分开。判决函数为：

$$d_{ij}(X) = W_{ij}^T X \quad . \quad \text{这里 } d_{ji} = -d_{ij} .$$

判别函数性质：

$$d_{ij}(X) > 0, \quad \forall j \neq i; \quad i, j = 1, 2, \dots, M, \quad \text{若 } X \in \omega_i$$

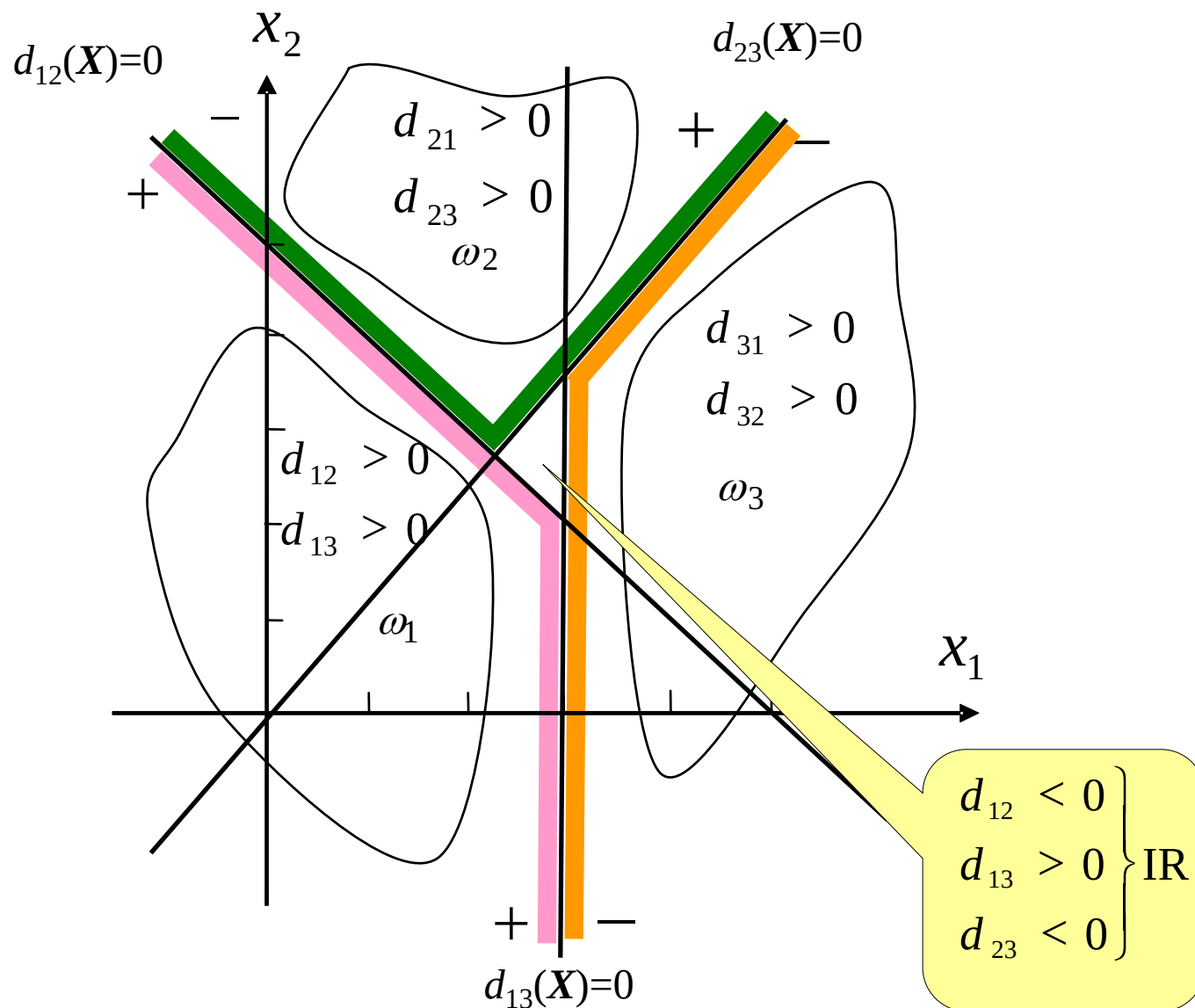
识别分类时：

在 M 类模式中，以 i 开始的 $M-1$ 个判决函数全为正时， $X \in \omega_i$ 。其中若有一个为负，则为IR区。

能用本方法分类的模式集称为：**成对线性可分的**。

如：对一个三类问题，如果 $d_{12}(X) > 0$ $d_{13}(X) > 0$

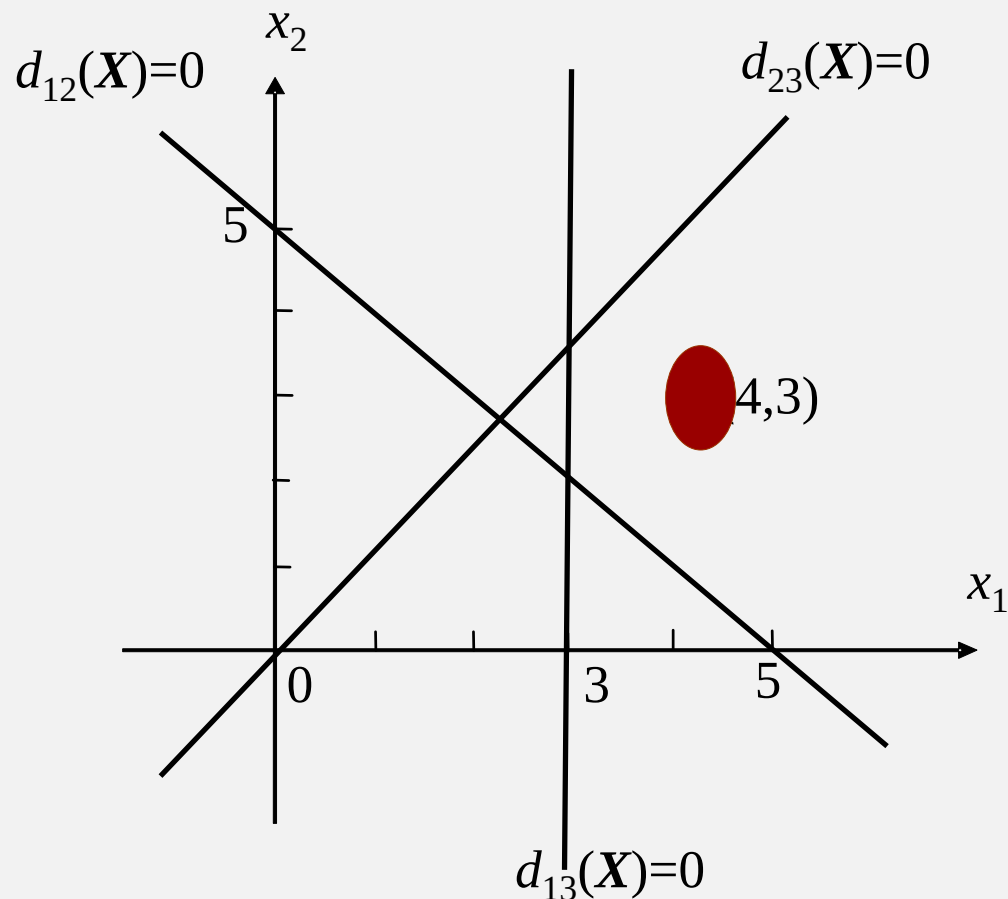
则 $X \in \omega_1$ 类，而 $d_{23}(X)$ 在判别 ω_1 类模式时不起作用。



例3 一个二维三类问题，已知三个判决函数为：

$$d_{12}(X) = -x_1 - x_2 + 5 \quad d_{13}(X) = -x_1 + 3 \quad d_{23}(X) = -x_1 + x_2$$

问模式 $X = [4, 3]^T$ 属于哪类？



解1 先计算： $d_{12}(X) = -2,$

$$d_{13}(X) = -1,$$

$$d_{23}(X) = -1$$

然后求： $d_{21}(X) = 2,$

$$d_{31}(X) = 1,$$

最后判： $d_{32}(X) = 1$

$$\left. \begin{array}{l} d_{31}(X) > 0 \\ d_{32}(X) > 0 \end{array} \right\} \Rightarrow X = [4, 3]^T \in \omega_3$$

与 $d_{12}(X)$ 值无关。

(3) 多类情况3: ω_i/ω_j 成对两分法 (无IR区)

当 ω_i/ω_j 成对两分法中的判别函数 $d_{ij}(X)$, 如果可以分解为

$$d_{ij}(X) = d_i(X) - d_j(X)$$

时, 那么 $d_i(X) > d_j(X)$ 就相当于多类情况2中的 $d_{ij}(X) > 0$ 。

此时, 对具有判别函数

$$d_i(X) = W_i^T X, \quad i = 1, \dots, M$$

的 M 类情况, 等价的判别函数性质为:

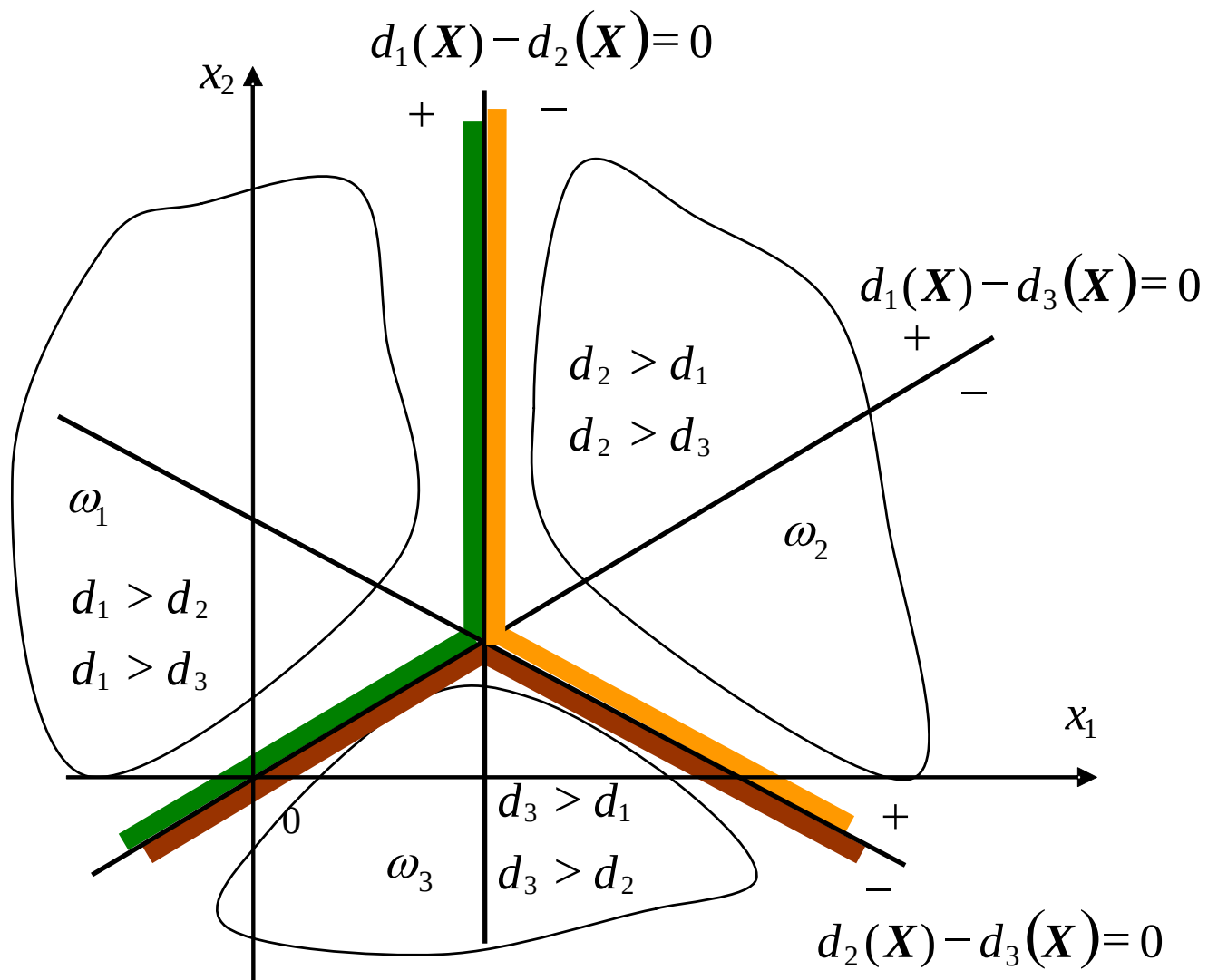
$$d_i(X) > d_j(X), \quad \forall j \neq i; \quad i, j = 1, 2, \dots, M, \quad \text{若 } X \in \omega_i$$

或:

$$d_i(X) = \max\{d_k(X), \quad k = 1, \dots, M\}, \quad \text{若 } X \in \omega_i$$

能用本方法分类的模式集称为: **线性可分的**。

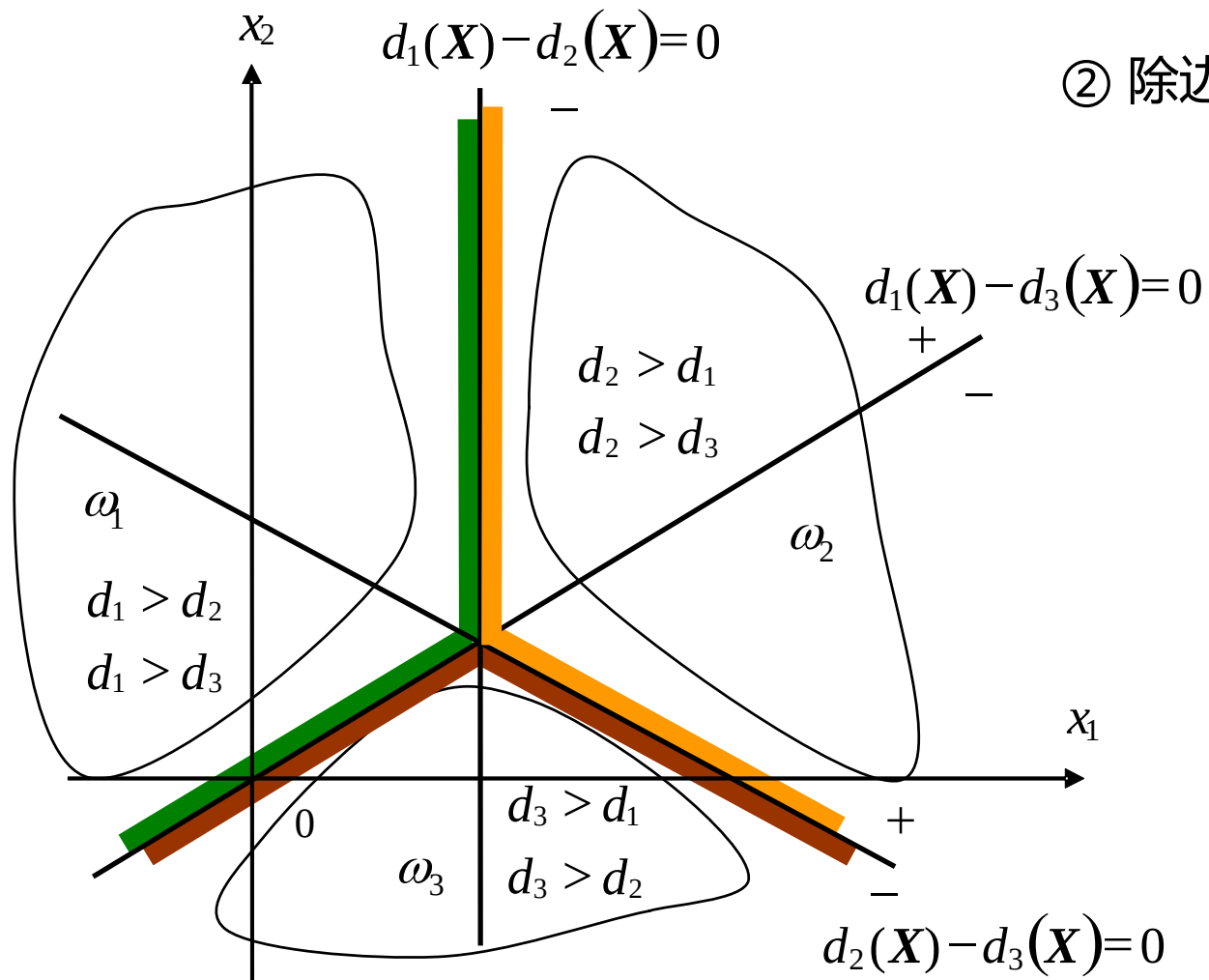
注意: 与前两种方法不同, 这里是**以大小进行判定的**, 不是仅仅以正负进行判定的。



识别分类时:

判别界面做差值。
对 ω_i 类, 应满足:
 $d_i >$ 其他所有 d_j

① 这是第二种情况的特例。因 $d_{ij}(\mathbf{X}) = d_i(\mathbf{X}) - d_j(\mathbf{X})$ ，若在第三种情况下可分（线性可分），则在第二种情况下也可分（成对线性可分），反之不一定成立。



② 除边界区外，没有不确定区域。

③ 把 M 类情况分成了 $(M-1)$ 个两类问题。并且 ω_i 类的判别界面全部与 $\omega_j (\forall j \neq i)$ 类的判别界面相邻（向无穷远处延伸的区域除外）。



例 5 一个二维三类模式分类器，满足多类情况3（成对两分法特例）要求的三个判决函数为：

$$d_1(\mathbf{X}) = -x_1 + x_2$$

$$d_2(\mathbf{X}) = x_1 + x_2 - 1$$

$$d_3(\mathbf{X}) = -x_2$$

试判断 $\mathbf{X}_0 = [1, 1]^T$ 属于哪一类，且分别给出三类的判决界面。

解：①

$$\left. \begin{array}{l} d_1(\mathbf{X}_0) = -1 + 1 = 0 \\ d_2(\mathbf{X}_0) = 1 + 1 - 1 = 1 \\ d_3(\mathbf{X}_0) = -1 \end{array} \right\} \Rightarrow \left. \begin{array}{l} d_2(\mathbf{X}_0) > d_1(\mathbf{X}_0) \\ d_2(\mathbf{X}_0) > d_3(\mathbf{X}_0) \end{array} \right\} \Rightarrow \mathbf{X}_0 \in \omega_2$$

② ω_1 类的判决函数:

$$d_1(\mathbf{X}) - d_2(\mathbf{X}) = -2x_1 + 1$$

$$d_1(\mathbf{X}) - d_3(\mathbf{X}) = -x_1 + 2x_2$$

ω_2 类的判决函数:

$$\begin{aligned} d_2(\mathbf{X}) - d_1(\mathbf{X}) \\ = 2x_1 - 1 \end{aligned}$$

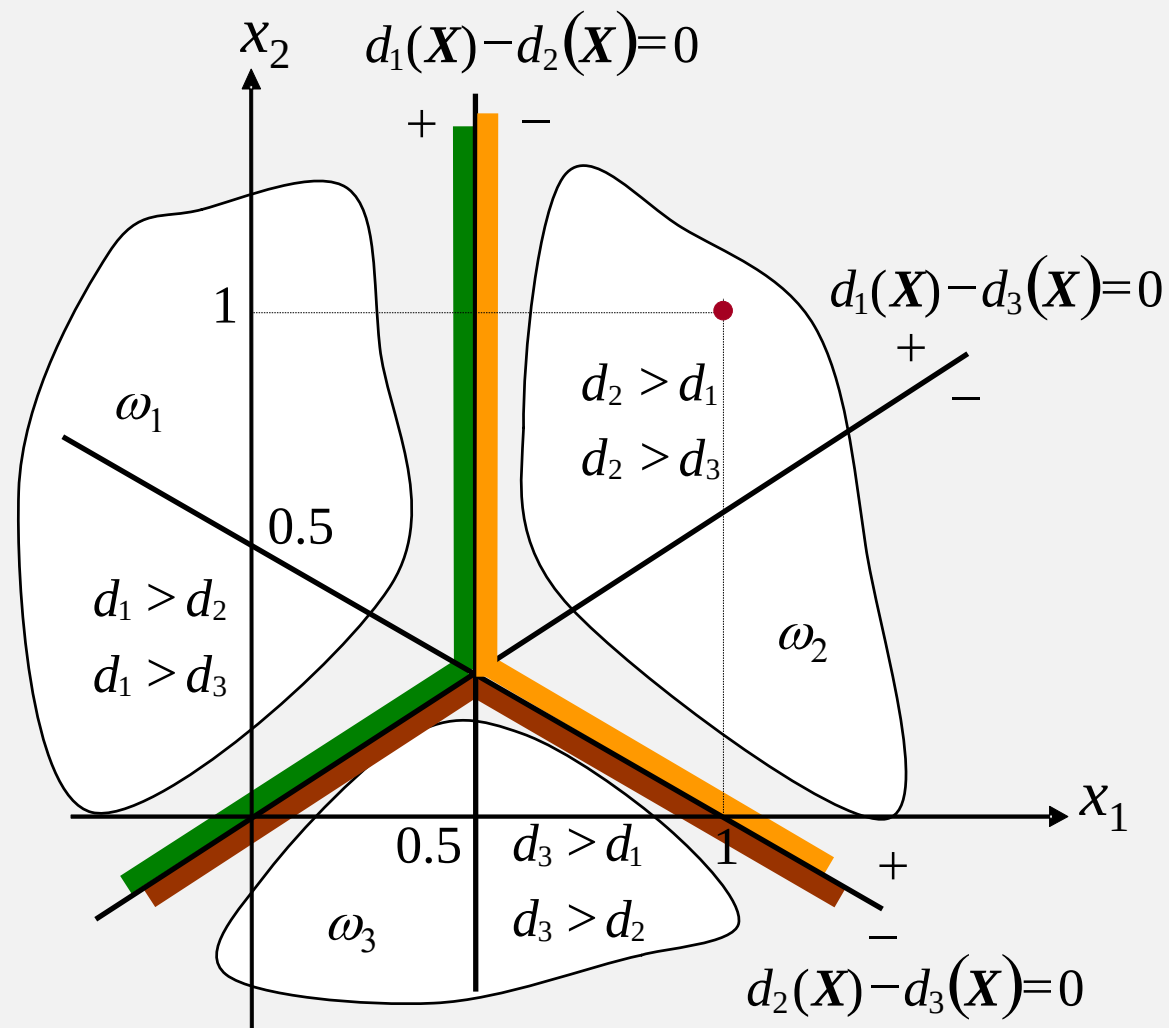
$$\begin{aligned} d_2(\mathbf{X}) - d_3(\mathbf{X}) \\ = x_1 + 2x_2 - 1 \end{aligned}$$

ω_3 类的判决函数:

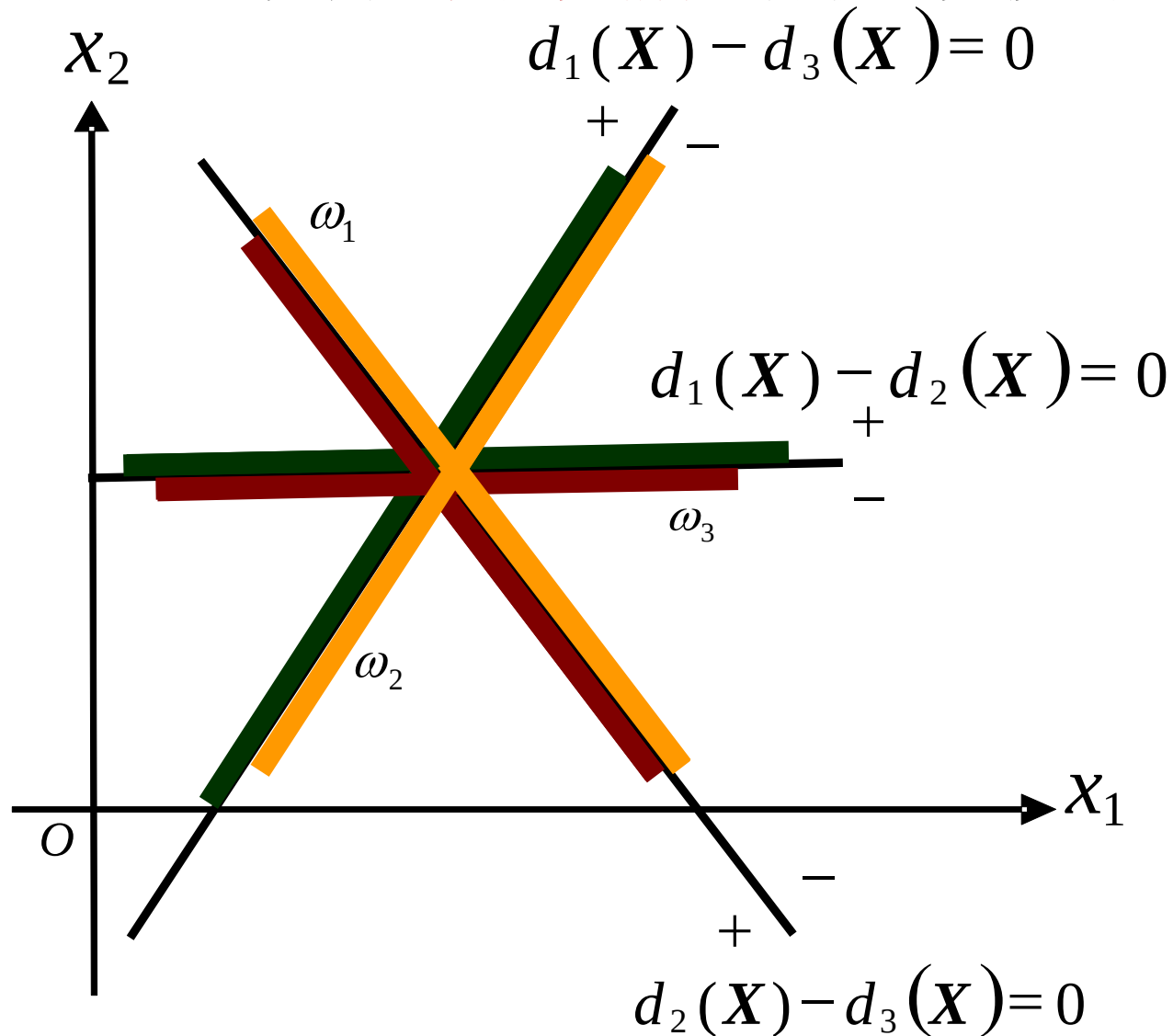
$$d_{31}(\mathbf{X}) = -d_{13}(\mathbf{X})$$

$$d_{32}(\mathbf{X}) = -d_{23}(\mathbf{X})$$

判决界面方程如图。



例 6 已知判决界面的位置和正负侧，按多类情况3分析三类模式的分布区域。





3. 小结

(1) 明确概念：线性可分。

一旦线性判别函数的系数 W_k 被确定以后，这些函数就可以作为模式分类器，对未知模式进行分类。

(2) $\omega_i/\bar{\omega}_i$ 与 ω_i/ω_j 两分法的比较：

对于 M 类模式的分类， $\omega_i/\bar{\omega}_i$ 是非两分法共需要 M 个判别函数， ω_i/ω_j 成对两分法需要 $M(M-1)/2$ 个。当 $M > 3$ 时，后者需要更多的判别式（缺点），但对模式集进行线性可分的可能性要更大一些（优点）。

原因：

一种类别模式 ω_j 的分布要比 $M-1$ 类模式 $\bar{\omega}_i$ 的分布更为聚集，因此 ω_i/ω_j 两分法受到的限制比 $\omega_i/\bar{\omega}_i$ 少，因此线性可分可能性大。



对线性判别函数，当模式**维数**已知时，判别函数的形式实际上已经确定，如：三维时

$$d(\mathbf{X}) = w_1x_1 + w_2x_2 + w_3x_3 + w_4 = \mathbf{W}^T \mathbf{X}$$

$$\mathbf{X} = [x_1, x_2, x_3, 1]^T \quad \mathbf{W} = [w_1, w_2, w_3, w_4]^T$$

只要求出权向量，分类器的设计即告完成。**本节**介绍通过梯度算法，利用已知类别的模式样本**训练权向量** $\mathbf{W}$ 。

(一) 梯度求解算法的基本原理

1. 梯度概念

设函数 $f(\mathbf{Y})$ 是向量 $\mathbf{Y} = [y_1, y_2, \dots, y_n]^T$ 的函数, 则 $f(\mathbf{Y})$ 的梯度定义为:

$$\nabla f(\mathbf{Y}) = \frac{d}{d\mathbf{Y}} f(\mathbf{Y}) = \left[\frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial y_2}, \dots, \frac{\partial f}{\partial y_n} \right]^T$$

梯度向量 最重要的性质之一:

当函数 f 的自变量沿梯度变化时, 函数的增长最快

即:

梯度的**方向**是函数 $f(\mathbf{Y})$ 在 $\mathbf{Y}$ 点增长最快的方向,

梯度的**模**是 $f(\mathbf{Y})$ 在增长最快的方向上的增长率 (增长率最大值)

显然: **负梯度指出了最陡下降方向**——梯度算法的依据。



2. 梯度算法

设两个线性可分的模式类 ω_1 和 ω_2 的样本共 N 个， ω_2 类样本乘(-1)。将两类样本分开的判决函数 $d(\mathbf{X})$ 应满足：

$$d(\mathbf{X}_i) = \mathbf{W}^T \mathbf{X}_i > 0 \quad i = 1, 2, \dots, N \quad \text{——} N \text{个不等式}$$

梯度算法的目的仍然是求一个满足上述条件的权向量，主导思想是将联立不等式求解 $\mathbf{W}$ 的问题，转换成求准则函数极小值的问题。

用负梯度向量的值对权向量 $\mathbf{W}$ 进行修正，实现使准则函数达到极小值的目的。

准则函数的选取原则：

具有唯一的最小值，并且这个最小值发生在 $\mathbf{W}^T \mathbf{X}_i > 0$ 时。



基本思路:

定义一个对错误分类敏感的准则函数 $J(W, X)$ ，在 J 的梯度方向上对权向量进行更新，从 $W(k)$ 计算 $W(k+1)$ 的递推关系为：

$$W(k+1) = W(k) + c(-\nabla J) = W(k) - c\nabla J$$

$$W(k+1) = W(k) - c \left[\frac{\partial J(W, X)}{\partial W} \right]_{W=W(k)}$$

其中 c 是固定的、正的比例因子。

梯度法求解步骤:

(1) 将样本写成规范化增广向量形式，选择准则函数，设置初始权向 $W(1)$ ，括号内为迭代次数 $k=1$ 。



(2) 依次输入训练样本 $\mathbf{X}$ 。设第 k 次迭代时输入样本为 $\mathbf{X}_i$ ，此时已有权向量 $\mathbf{W}(k)$ ，求 $\nabla J(k)$ ：

$$\nabla J(k) = \left. \frac{\partial J(\mathbf{W}, \mathbf{X}_i)}{\partial \mathbf{W}} \right|_{\mathbf{W}=\mathbf{W}(k)}$$

权向量修正为：

$$\mathbf{W}(k+1) = \mathbf{W}(k) - c \nabla J(k)$$

迭代次数 $k+1$ ，输入下一个训练样本，计算新的权向量，直至对全部训练样本完成一轮迭代。

(3) 在每一轮迭代中，如果有一个样本使 $\nabla J \neq 0$ ，回到(2)进行下一轮迭代。否则， $\mathbf{W}$ 不再变化，算法收敛。

例子 选择准则函数, $J(W, X) = |W^T X| - W^T X$ 简单地考虑 X 为一维增广模式的情况 $X=[0,1]$, 此时 $W=[w_1 \ w_2]$, 因为 X 的第1项为0, 简记为**标量**:

$$J(W, X) = |w| - w$$

如果**当前的 W 使得当前的 X 被错分**, 即:

$$W^T X < 0 \Rightarrow w \cdot 1 < 0 \Rightarrow w < 0, \text{ 则此时有}$$

$$\nabla J = \frac{\partial J}{\partial W}(W, X) \Big|_{X=1} = \frac{\partial}{\partial w} (|w| - w) = \frac{\partial}{\partial w} (-2w) = -2$$

此时的更新方程是 $W(k+1) = W(k) - c \cdot (-2) = W(k) + 2c$

而在**正确分类**时:

$$W(k+1) = W(k) - c \nabla J$$

$$W^T X > 0 \Rightarrow w \cdot 1 > 0 \Rightarrow w > 0 \quad \nabla J = \frac{\partial}{\partial w} (|w| - w) = 0$$

$\therefore W(k+1) = W(k) - c \cdot 0 = W(k)$, 对权向量不做修正。



说明:

$$a) \quad \mathbf{W}(k+1) = \mathbf{W}(k) - c \nabla J = \mathbf{W}(k) - c \left[\frac{\partial J(\mathbf{W}, \mathbf{X})}{\partial \mathbf{W}} \right]_{\mathbf{W}=\mathbf{W}(k)}$$

随着权向量 $\mathbf{W}$ 向理想值接近, 准则函数关于 $\mathbf{W}$ 的导数越来越趋近于0, 这意味着准则函数 J 越来越接近最小值。当最终 $\nabla J = 0$ 时, J 达到最小值, $\mathbf{W}$ 不再改变, 算法收敛。

b) c 值的选择很重要, 如 c 值太小, 收敛太慢; 但若太大, 搜索又可能过头, 甚至引起发散。一般, 开始阶段 c 取大一些, 收尾阶段取小——变步长。

c) 梯度算法是求解权向量的一般解法, 算法的具体计算形式取决于准则函数 $J(\mathbf{W}, \mathbf{X})$ 的选择, $J(\mathbf{W}, \mathbf{X})$ 的形式不同, 得到的具体算法不同。



(二) 梯度求解法的一个实现——固定增量法

准则函数: $J(W, X) = \frac{1}{2} (|W^T X| - W^T X)$

该准则函数有唯一最小值 “0”, 且发生在 $W^T X > 0$ 的时候。

求 $W(k)$ 的递推公式:

设 $X = [x_1, x_2, \dots, x_n, 1]^T$ $W = [w_1, w_2, \dots, w_n, w_{n+1}]^T$

1. 求 J 的梯度 $\nabla J = \frac{\partial J(W, X)}{\partial W} = ?$

方法: 函数对向量求导=函数对向量的分量求导, 即

$$\frac{\partial f}{\partial X} = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right]^T$$



①首先求 $\mathbf{W}^T \mathbf{X}$ 部分:

$$\frac{\partial(\mathbf{W}^T \mathbf{X})}{\partial \mathbf{W}} = \frac{\partial}{\partial \mathbf{W}} \left(\sum_{i=1}^n w_i x_i + w_{n+1} \right)$$

$$= \left[\frac{\partial}{\partial w_1} \left(\sum_{i=1}^n w_i x_i + w_{n+1} \right), \dots, \frac{\partial}{\partial w_k} \left(\sum_i w_i x_i + w_{n+1} \right), \dots, \frac{\partial}{\partial w_{n+1}} \left(\sum_i w_i x_i + w_{n+1} \right) \right]^T$$

$$= [x_1, \dots, x_k, \dots, x_n, 1]^T = \mathbf{X}$$

$$J(\mathbf{W}, \mathbf{X}) = \frac{1}{2} (\|\mathbf{W}^T \mathbf{X}\| - \mathbf{W}^T \mathbf{X})$$

另：矩阵论中有（也可以直接按梯度定义演算）

$$\frac{d\mathbf{X}}{d\mathbf{X}^T} = \frac{d\mathbf{X}^T}{d\mathbf{X}} = \mathbf{I}_{n \times n}$$

$$\therefore \frac{\partial(\mathbf{W}^T \mathbf{X})}{\partial \mathbf{W}} = \mathbf{I}_{(n+1) \times (n+1)} \mathbf{X}_{(n+1) \times 1} = \mathbf{X}_{(n+1) \times 1}$$

② 由①的结论 $\frac{\partial(\mathbf{W}^T \mathbf{X})}{\partial \mathbf{W}} = \mathbf{X}$ 有:

$$\mathbf{W}^T \mathbf{X} > 0 \text{ 时, } \frac{\partial(|\mathbf{W}^T \mathbf{X}|)}{\partial \mathbf{W}} = \frac{\partial(\mathbf{W}^T \mathbf{X})}{\partial \mathbf{W}} = \mathbf{X}$$

$$\mathbf{W}^T \mathbf{X} \leq 0 \text{ 时, } \frac{\partial(|\mathbf{W}^T \mathbf{X}|)}{\partial \mathbf{W}} = \frac{\partial(-\mathbf{W}^T \mathbf{X})}{\partial \mathbf{W}} = -\mathbf{X}$$

$$\therefore \frac{\partial(|\mathbf{W}^T \mathbf{X}|)}{\partial \mathbf{W}} = [\text{sgn}(\mathbf{W}^T \mathbf{X})] \cdot \mathbf{X}$$

$$\text{其中 } \text{sgn}(\mathbf{W}^T \mathbf{X}) = \begin{cases} +1, & \text{若 } \mathbf{W}^T \mathbf{X} > 0 \\ -1, & \text{若 } \mathbf{W}^T \mathbf{X} \leq 0 \end{cases}$$

$$\therefore \nabla J = \frac{\partial J(\mathbf{W}, \mathbf{X})}{\partial \mathbf{W}} = \frac{1}{2} [\mathbf{X} \text{sgn}(\mathbf{W}^T \mathbf{X}) - \mathbf{X}]$$

$$J(\mathbf{W}, \mathbf{X}) = \frac{1}{2} (|\mathbf{W}^T \mathbf{X}| - \mathbf{W}^T \mathbf{X})$$

2. 求 $W(k+1)$

将 $\nabla J = \frac{\partial J(W, X)}{\partial W} = \frac{1}{2} [X \operatorname{sgn}(W^T X) - X]$ 代入

$$W(k+1) = W(k) - c \nabla J = W(k) - c \left[\frac{\partial J(W, X)}{\partial W} \right]_{W=W(k)}$$

$$\text{得: } W(k+1) = W(k) - c \frac{1}{2} [X \operatorname{sgn}(W^T(k) X) - X]$$

$$= W(k) + \frac{c}{2} [X - X \operatorname{sgn}(W^T(k) X)]$$

$$= W(k) + \begin{cases} 0, & \text{若 } W^T(k) X > 0 \\ cX, & \text{若 } W^T(k) X \leq 0 \end{cases}$$



$$\text{即: } \mathbf{W}(k+1) = \mathbf{W}(k) + \begin{cases} 0, & \text{若 } \mathbf{W}^T(k)\mathbf{X} > 0 \\ c\mathbf{X}, & \text{若 } \mathbf{W}^T(k)\mathbf{X} \leq 0 \end{cases}$$

即为**固定增量算法**，与**感知器算法**（神经网络）完全相同。

$$\mathbf{W}(k+1) = \begin{cases} \mathbf{W}(k), & \text{若 } \mathbf{W}^T(k)\mathbf{X}_i > 0 \\ \mathbf{W}(k) + c\mathbf{X}_i, & \text{若 } \mathbf{W}^T(k)\mathbf{X}_i \leq 0 \end{cases}$$

可以看出，神经网络中的**感知器算法是梯度方法的特例**。梯度法是将感知器算法中联立不等式求解 $\mathbf{W}$ 的问题，转换为求目标函数极小值的问题。

分类算法是通过模式**样本**来确定判别函数的系数，因此必须采用有**代表性**的数据，这样训练出来的判别函数，能合理反映模式数据的区隔情况。



线性回归：基于线性模型的回归，相应的线性模型称为线性回归模型。一般而言，解释变量和响应变量都是**连续值**。

对于任意给定的样本 $\mathbf{x}$ ，线性回归模型表示为：

$$f(\mathbf{x}) = w_1x_1 + w_2x_2 + \cdots + w_mx_m$$

其中 $\mathbf{w} = (w_1, w_2, \dots, w_m)^T$ 为**参数向量**。

线性回归与线性分类问题非常相似，只不过线性回归的响应变量一般是**定量型**，而线性分类涉及的响应变量主要是**定性型**，比如：有病 vs 无病。穷人 vs 富人。喜欢看武打片 vs 喜欢看言情片。为了适合计算，把这些定性变量用 $\{0, 1\}$ 或 $\{-1, +1\}$ 代替。

使用符号函数作为判决函数：简单观察可知，前面讨论线性判别分类时，实际上是在线性回归函数 $f(\mathbf{x})$ （也即 $d(\mathbf{X})$ ）上，复合了一个符号型判决函数：

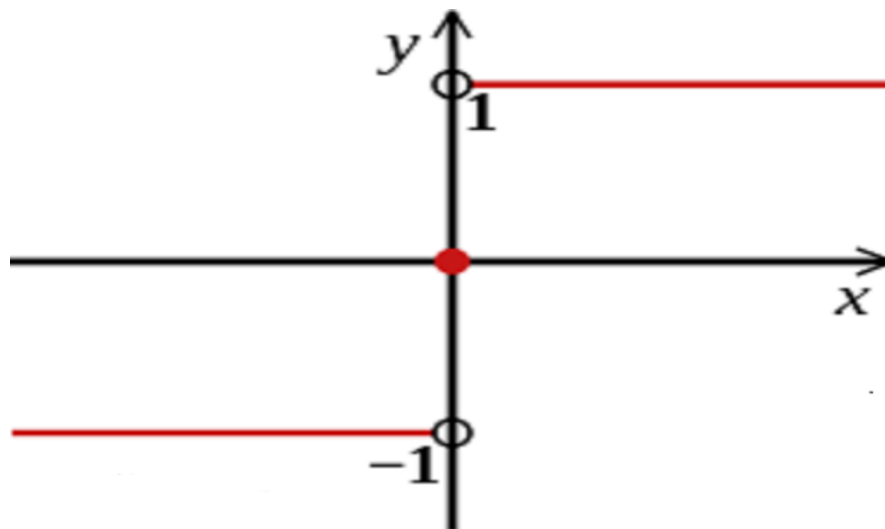
$$y = \text{sign}(f(\mathbf{x})) = \text{sign}(w_1x_1 + w_2x_2 + \cdots + w_mx_m)$$

这个判决函数非常硬——只看是否大于零，不管与零有多远！

我们如何利用成熟的线性回归方法进行更细致的分类呢？

我们能不能更多地利用判决函数值的大小信息、而不仅仅是利用其正负信息呢？

$$\text{sign}(x) = \begin{cases} 1, & x > 0 \\ 0, & x = 0 \\ -1, & x < 0 \end{cases}$$



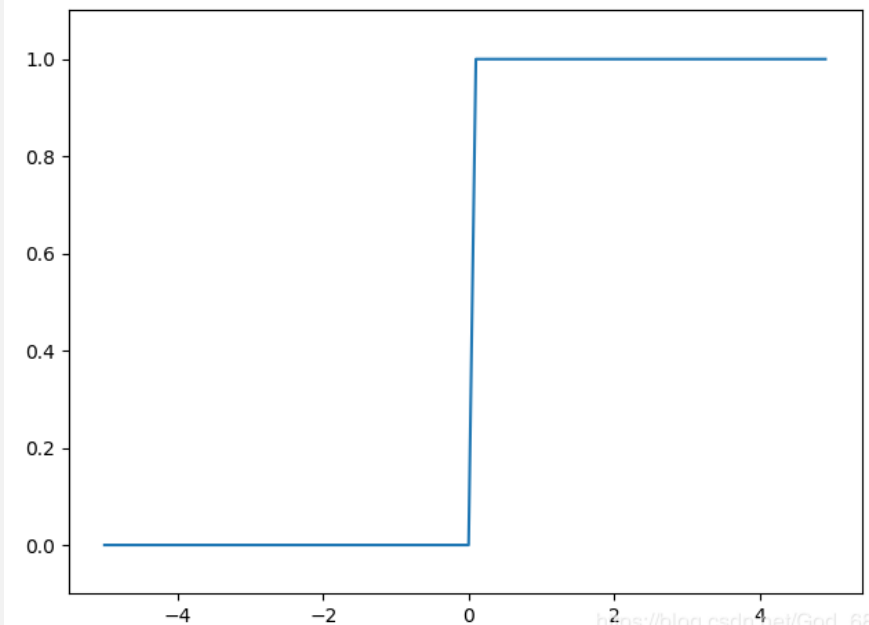


使用跃阶函数作为判决函数：当类标记从 $\{1, -1\}$ 改为 $\{1, 0\}$ 时，相当于把判决函数改为阶跃函数（step function）

$$\text{step}(x) = \frac{1}{2}(\text{sign}(x) + 1)$$

当 $f(x)$ 大于零时将样本 x 划分为正类；小于零时将 x 划分为负类；等于零时将 x 随机划分。

$$\text{step}(f(x)) = \begin{cases} 0 & f(x) < 0; \\ 0.5 & f(x) = 0; \\ 1 & f(x) > 0; \end{cases}$$

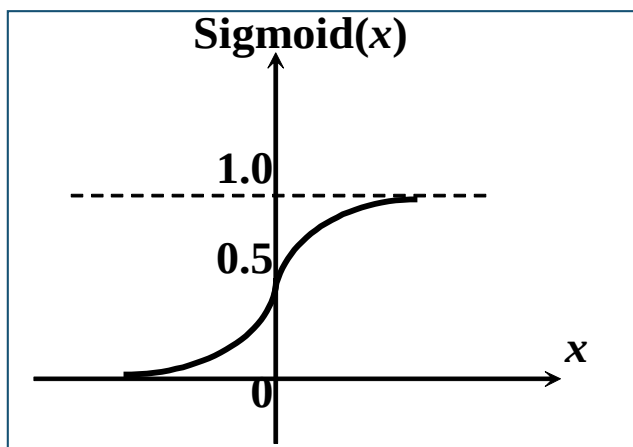


使用Sigmoid作为分类判决函数：具有良好数学性质的判别函数，克服了跃阶函数的不连续性，实现对连续值的离散化

$$g(f(x)) = \text{Sigmoid}(f(x)) = \frac{1}{1+e^{-w^T x}}$$

令 $H(x) = \text{Sigmoid}(f(x))$ ， $H(x)$ 值域为 $(0, 1)$ ，可将 $H(x)$ 看成是一个关于 x 的概率分布，即：

- 若 $H(x)$ 的值越接近1，则 x 属于正类的可能性就越大
- 若 $H(x)$ 的值越接近0，则 x 属于正类的可能性就越小



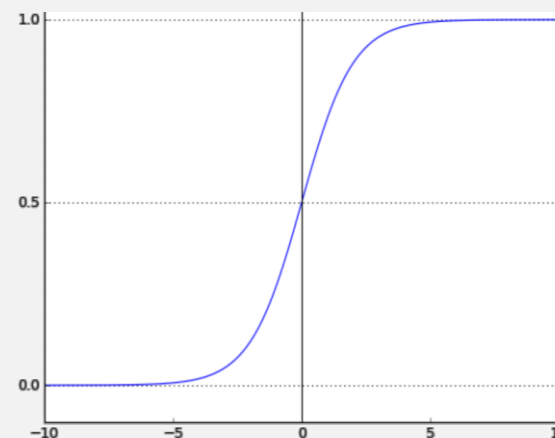


1 对数几率回归的模型

- 给定数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ ，样本空间由 d 维向量张成， $x_i = (x_{i1}, x_{i2}, \dots, x_{id})^T$ ， $y_i \in \{c_1 = 1, c_0 = 0\}$
- 找一个单调可微的**联系函数**，将样本 x_i 的线性回归预测值 z_i 与样本的分类**后验概率** $\hat{y}_i = P(c_1 | x_i)$ 联系起来，构成一个线性分类器：即对样本 x_i 与其分类后验概率 $\hat{y}_i = P(c_1 | x_i)$ 做**广义线性回归**，选择 Logistic 函数作为联系函数

- Logistic 函数是一种形为 S (Sigmoid) 的函数，由皮埃尔·弗朗索瓦·韦吕勒 (Pierre-François Verhulst) 在研究人口增长时提出：

$$f(x) = \frac{1}{1 + e^{-x}}$$



- LR模型的设计思想:

$$\mathbf{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{id} \end{bmatrix} \xrightarrow{\text{①线性回归}} z_i = \mathbf{x}_i^T \mathbf{w} + w_0 \xrightarrow{\text{②Logistic映射}} \hat{y}_i = \frac{1}{1+e^{-z_i}} = \frac{1}{1+e^{-(\mathbf{x}_i^T \mathbf{w} + w_0)}} = P(c_1 | X = \mathbf{x}_i)$$

$$\text{满足: } \begin{cases} z_i > 0 & \hat{y}_i = P(c_1 | \mathbf{x}_i) > 0.5 \\ z_i = 0 & \hat{y}_i = P(c_1 | \mathbf{x}_i) = 0.5 \\ z_i < 0 & \hat{y}_i = P(c_1 | \mathbf{x}_i) < 0.5 \end{cases}$$

如样本 $\mathbf{x}_i$ 属于正例, 则 $z_i > 0$; $\mathbf{x}_i$ 属于负例
则 $z_i < 0$; 样本在决策界上, 则 $z_i = 0$



- 在贝叶斯分类器中, $P(c_1|\mathbf{x})=P(c_0|\mathbf{x})$ 是二分类的分类决策界, 则:

- 决策界满足:

$$\ln \frac{P(c_1|\mathbf{x})}{P(c_0|\mathbf{x})} = \ln \frac{\hat{y}}{1-\hat{y}} = 0$$

则: $\ln \frac{\hat{y}}{1-\hat{y}} = \ln \frac{1}{\frac{1+e^{-z}}{e^{-z}}} = \ln e^z = z = 0$ 即 $z = \mathbf{x}^T \mathbf{w} + w_0 = 0$ 是决策界方程

- 同时 $z = \ln \frac{\hat{y}}{1-\hat{y}}$ 是 $\mathbf{x}$ 作为正例与反例的概率的比值的对数, 这个比值又被称为“几率”, 这也就是模型被称为“对数几率回归”的由来

2 对数几率回归的学习准则

➤ LR模型的学习准则可以从**最大似然**估计导出，用于衡量模型预测值与真实值之间的差异

- 考虑让样本 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$ $y_i \in \{0, 1\}$ 中每个样本 $(\mathbf{x}_i, y_i)$ 属于其真实标记的概率越大越好，构造似然函数：

$$l(\mathbf{w}) = \prod_{i=1}^m [P(\mathbf{x}_i)]^{y_i} [1 - P(\mathbf{x}_i)]^{1-y_i} \quad \leftarrow \text{伯努利型的似然估计}$$

$$\text{其中 } P(\mathbf{x}) = P(c_1|\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{x}^T \mathbf{w} + w_0)}} = \frac{e^{(\mathbf{x}^T \mathbf{w} + w_0)}}{1 + e^{(\mathbf{x}^T \mathbf{w} + w_0)}}$$

- 则其对数似然函数有：

$$ll(\mathbf{w}) = \ln l(\mathbf{w}) = \sum_{i=1}^m [y_i \cdot \ln P(\mathbf{x}_i) + (1 - y_i) \cdot \ln(1 - P(\mathbf{x}_i))]$$



- LR模型的优化目标函数可以写为：

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmax}} ll(\mathbf{w}) = \underset{\mathbf{w}}{\operatorname{argmax}} \left[\sum_{i=1}^m [y_i \cdot \ln P(\mathbf{x}_i) + (1 - y_i) \cdot \ln(1 - P(\mathbf{x}_i))] \right]$$

- 对上式取负号，可以得到等价的优化目标函数：

$$L(\mathbf{w}) = -ll(\mathbf{w}) = \sum_{i=1}^m -y_i \cdot \ln P(\mathbf{x}_i) - (1 - y_i) \cdot \ln(1 - P(\mathbf{x}_i))$$
$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} L(\mathbf{w}) = \underset{\mathbf{w}}{\operatorname{argmin}} \left[\sum_{i=1}^m -y_i \cdot \ln P(\mathbf{x}_i) - (1 - y_i) \cdot \ln(1 - P(\mathbf{x}_i)) \right]$$

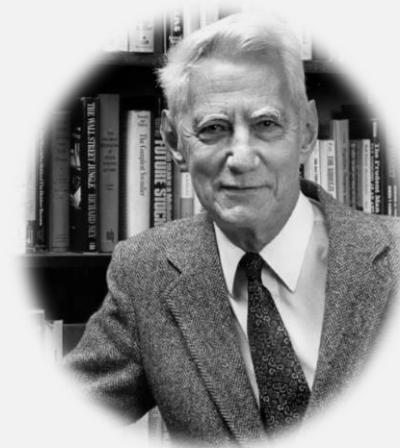
- 其中 $L(\mathbf{w})$ 函数即LR模型的代价（损失）函数，又叫“**二分类交叉熵**（corss-entropy）”损失函数

3 交叉熵损失

- 用交叉熵（互熵）定义损失函数经常用于分类问题中，在对数几率回归以及神经网络中会大量使用到



- 克劳德·艾尔伍德·香农（Claude Elwood Shannon, 1916—2001），出生于美国密歇根州佩托斯基，美国数学家、发明家、密码学家，信息论创始人，美国国家工程院院士、美国国家科学院院士、美国艺术与科学院院士，麻省理工学院名誉教授。香农提出了信息熵的概念，为信息论和数字通信奠定了基础



克劳德·艾尔伍德·香农

Claude Elwood Shannon

(1916.4—2001.2)



- 根据香农信息论，设随机事件 X 有 k 个可能取值（对应 k 分类），其中 $X = x_k$ 的信息量：

$$I(x_k) = -\log P(X = x_k)$$

- 信息熵（entropy）是信息量函数的期望。信息熵越大表明信息越不确定（越丰富），信息熵越小表明信息越确定（越枯燥）

$$\text{Ent}(X) = -\sum_{i=1}^k P(X = x_i) \log P(X = x_i)$$

- 随机事件 X 有 k 个可能取值，其真实分布 $P(X)$ 与近似分布 $Q(X)$ 的关系可以用交叉熵来刻画：

$$\text{Ent}(P, Q) = -\sum_{i=1}^k P(X = x_i) \log Q(X = x_i)$$



➤ 对于样本 $\mathbf{x}_i$, 其标记值为 y_i , 预测值为 $\hat{y}_i$, 其二分类交叉熵:

- 其二分类交叉熵:

$$\begin{aligned} & -y_i \cdot \ln P(\mathbf{x}_i) - (1 - y_i) \cdot \ln(1 - P(\mathbf{x}_i)) \\ & = -y_i \cdot \ln \hat{y}_i - (1 - y_i) \cdot \ln(1 - \hat{y}_i) \end{aligned} \quad \text{二分类, 写成两项和}$$

- 对于 m 个样本, 其代价函数:

$$\begin{aligned} L(\mathbf{w}) &= - \sum_{i=1}^m [y_i \cdot \ln P(\mathbf{x}_i) + (1 - y_i) \cdot \ln(1 - P(\mathbf{x}_i))] \\ &= - \sum_{i=1}^m \left[y_i \cdot \ln \frac{P(\mathbf{x}_i)}{1 - P(\mathbf{x}_i)} + \ln(1 - P(\mathbf{x}_i)) \right] \\ &= - \sum_{i=1}^m \left[y_i \cdot (\mathbf{x}_i^T \mathbf{w} + w_0) - \ln(1 + e^{\mathbf{x}_i^T \mathbf{w} + w_0}) \right] \end{aligned}$$



- 交叉熵损失的正则化与线性回归中MSE正则化类似

$$\text{代价函数 } L(\mathbf{w}) = - \sum_{i=1}^m \left[y_i \cdot (\mathbf{x}^T \mathbf{w} + w_0) - \ln(1 + e^{\mathbf{x}^T \mathbf{w} + w_0}) + \frac{\lambda}{2} \sum_{j=1}^d w_j^2 \right]$$

w 的L2范数的平方

λ 为正则化系数



例子. 假设一组训练样本的标记值为{1, 0, 1, 0, 1}, 分类器A的预测结果为{0.9, 0.01, 0.95, 0.03, 0.99}, 分类器B的预测结果为{0.85, 0.45, 0.75, 0.55, 0.7}, 两个分类器分别计算**平均交叉熵**:

A:
$$-[(1\ln 0.9 + 0\ln 0.1) + (0\ln 0.01 + 1\ln 0.99) + (1\ln 0.95 + 0\ln 0.05) + (0\ln 0.03 + 1\ln 0.97) + (1\ln 0.99 + 0\ln 0.01)] \times 0.2 = 0.207 \times 0.2 = 0.04$$

B:
$$-[(1\ln 0.85 + 0\ln 0.15) + (0\ln 0.45 + 1\ln 0.55) + (1\ln 0.75 + 0\ln 0.25) + (0\ln 0.55 + 1\ln 0.45) + (1\ln 0.7 + 0\ln 0.3)] \times 0.2 = 2.203 \times 0.2 = 0.44$$

上述两种情况对比, A的损失明显低于B的损失, 说明分类器A更优

注: 按照信息熵的定义, log应该取2为底。但机器学习中一般需要对交叉熵损失求导, 故log用ln代替



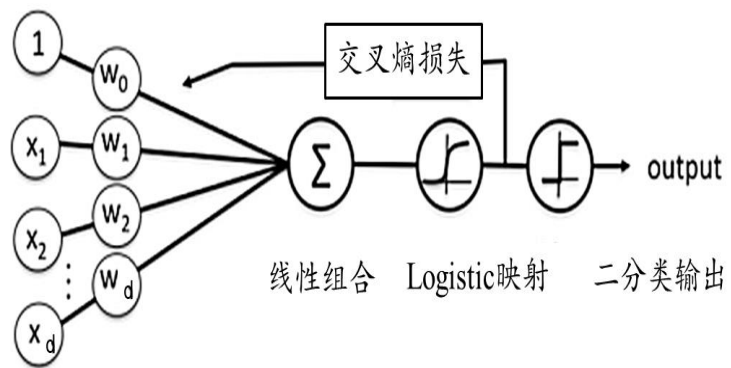
4 对数几率回归的优化算法

- 求解最小交叉熵损失函数
 - 梯度下降法 (Gradient descent method) (同前)
 - 牛顿迭代法 (Newton method) *

在对数几率回归中为什么选用交叉熵作为损失函数，而不选用MSE作为损失函数？

5 模型小结

模型



- 核心参数：
特征权向量 $\mathbf{w}$

策略（准则）

- 最小交叉熵损失：

$$\operatorname{argmin}_{\mathbf{w}} \left[\sum_{i=1}^m -y_i \cdot \ln P(\mathbf{x}_i) - (1 - y_i) \cdot \ln(1 - P(\mathbf{x}_i)) \right]$$

算法

- 梯度下降法：
$$\mathbf{w}^{(i+1)} \leftarrow \mathbf{w}^{(i)} - \eta \cdot \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}}$$



- 罗纳德·费希尔（1890~1962），英国统计与遗传学家，现代统计科学的奠基人之一，并对达尔文进化论作了基础澄清的工作。线性判别分析（Linear Discriminant Analysis, LDA）是一种经典的线性模型，在两分类问题上最早由费希尔在1936年提出，也被称为Fisher判别分析



罗纳德·费希尔
Ronald Fisher
(1890—1962)

- 线性判别分析（Linear Discriminant Analysis, LDA）模型是一种线性分类器模型。同时该模型也是一种有监督的**降维**模型



注意区分：在机器学习领域还有另外一个LDA，被称为隐迪利克雷分配（Latent Dirichlet Allocation），主要用于文档主题分析，现在也被广泛用于时空数据分析。它和线性判别分析都是机器学习的经典模型，方法及应用领域完全不同

1 线性判别分析的模型

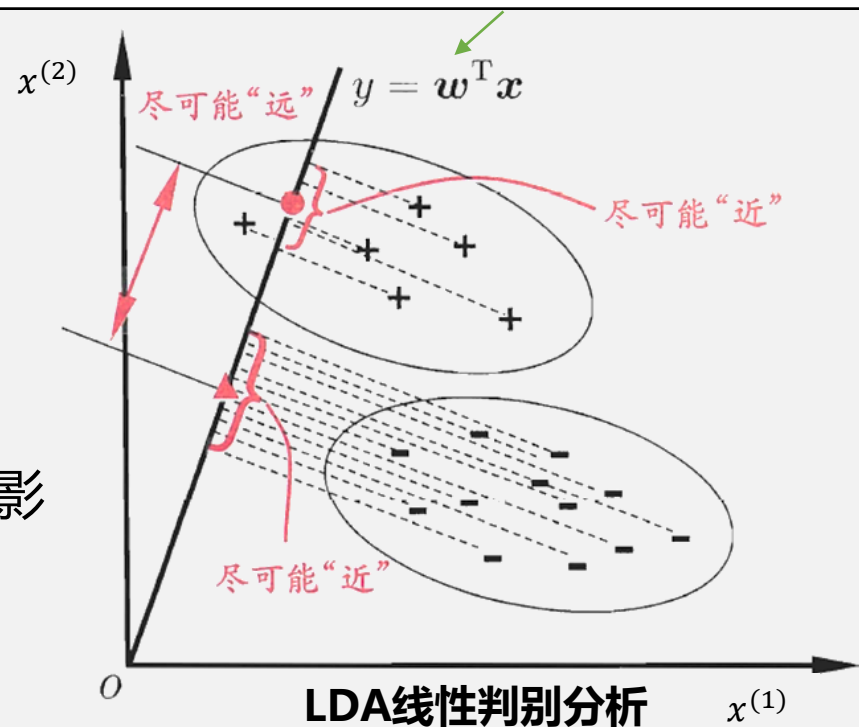
- LDA的**思想**：对于二分类问题，在给定的训练样集上，设法将样例投影到一条“直线”上：使得同类样例的投影点尽可能的近，异类样例的投影点尽可能的远离

- **作为特征提取模型：**

将 d 维度数据投影到 **$k-1$ 维**上， k 为样本的类别数

- **作为分类器：**

根据样本训练好这条“直线”，在预测阶段将新样本投影到该直线上，根据投影点的位置确定新样本的类别



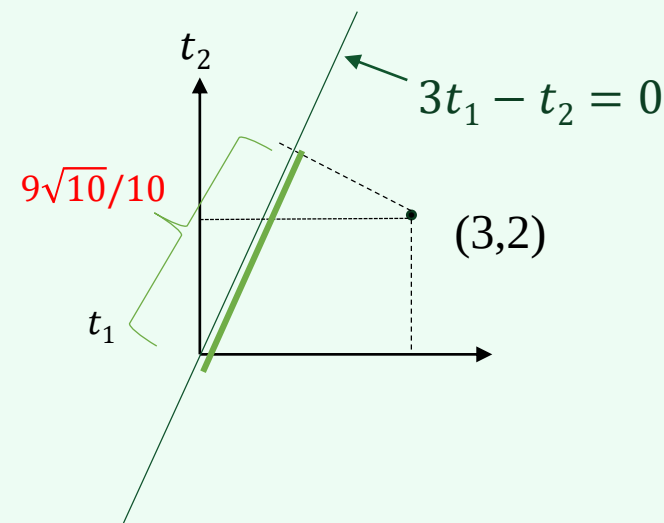


例子. 计算向量 $(3, 2)$ 在直线 $3t_1 - t_2 = 0$ 下的投影

解：由直线方程可知直线的方向向量 $\mathbf{w} = (1, 3)^T$ ，化为单位向量 $\mathbf{w}^T = (\sqrt{10}/10, 3\sqrt{10}/10)$ ，则其在直线下的投影为

$$\begin{pmatrix} \sqrt{10}/10 \\ 3\sqrt{10}/10 \end{pmatrix} \cdot \begin{pmatrix} 3 \\ 2 \end{pmatrix} = (\sqrt{10}/10, 3\sqrt{10}/10) \begin{pmatrix} 3 \\ 2 \end{pmatrix} = (9\sqrt{10}/10)$$

点乘



*注意： $9\sqrt{10}/10$ 是 t_1-t_2 坐标系下向量 $(3, 2)$ 在新空间 $\mathbf{w}$ 下的投影坐标。因新空间（直线）只有一个基向量，所以投影点为一个数（2维投影到1维）。这个数字其实也是 t_1-t_2 坐标系下投影向量 $(9/10, 27/10)$ 的长度



- 平面直线 $At_1 + Bt_2 + C = 0$ 的法向量为 (A, B) ，方向向量为 $(-B, A)$
- 在同一坐标系下，向量 $\vec{u}$ 在 **单位向量** $\vec{v}$ 的投影向量为 $(\vec{u} \cdot \vec{v}) \times \vec{v}$

➤ 考虑二分类, 样本 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$, $y_i \in \{0, 1\}$, 每个样本含有 d 维特征, $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{id})^T$

- 第 i 类样本的集合: X_i
- 第 i 类样本的均值向量: μ_i
- 第 i 类样本的协方差矩阵: Σ_i
- 两类样本的中心向量在直线 $\mathbf{w}$ 上的投影: $\mathbf{w}^T \mu_0$ 和 $\mathbf{w}^T \mu_1$ (直线 $\mathbf{w}$ 下的1维坐标)
- 两类样本在直线 $\mathbf{w}$ 上投影点的协方差: $\mathbf{w}^T \Sigma_0 \mathbf{w}$ 和 $\mathbf{w}^T \Sigma_1 \mathbf{w}$

$\mathbf{w}$ 为直线的方向向量



再次强调! m 是样本个数, d 是样本维数



2 线性判别分析的学习准则

➤ 考虑优化目标:

- 同类样例的投影点尽可能的近: 两个投影的协方差和尽可能小

$$\mathbf{w}^T \Sigma_0 \mathbf{w} + \mathbf{w}^T \Sigma_1 \mathbf{w}$$

- 异类样例投影点尽可能的远离: 各均值向量投影点距离尽可能大

$$\|\mathbf{w}^T \mu_0 - \mathbf{w}^T \mu_1\|_2^2 \quad \text{距离度量选用2范数平方}$$

➤ 构造目标函数:

$$J = \frac{\|\mathbf{w}^T \mu_0 - \mathbf{w}^T \mu_1\|_2^2}{\mathbf{w}^T \Sigma_0 \mathbf{w} + \mathbf{w}^T \Sigma_1 \mathbf{w}} = \frac{\mathbf{w}^T (\mu_0 - \mu_1)(\mu_0 - \mu_1)^T \mathbf{w}}{\mathbf{w}^T (\Sigma_0 + \Sigma_1) \mathbf{w}}$$



推导：

$$J = \frac{\|\mathbf{w}^T \boldsymbol{\mu}_0 - \mathbf{w}^T \boldsymbol{\mu}_1\|_2^2}{\mathbf{w}^T \boldsymbol{\Sigma}_0 \mathbf{w} + \mathbf{w}^T \boldsymbol{\Sigma}_1 \mathbf{w}} = \frac{\|(\mathbf{w}^T \boldsymbol{\mu}_0 - \mathbf{w}^T \boldsymbol{\mu}_1)^T\|_2^2}{\mathbf{w}^T (\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1) \mathbf{w}}$$

$$= \frac{\|(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T \mathbf{w}\|_2^2}{\mathbf{w}^T (\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1) \mathbf{w}} = \frac{[(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T \mathbf{w}]^T (\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T \mathbf{w}}{\mathbf{w}^T (\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1) \mathbf{w}}$$

$$= \frac{\mathbf{w}^T (\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1) (\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T \mathbf{w}}{\mathbf{w}^T (\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1) \mathbf{w}}$$

类内散度矩阵 S_w

类间散度矩阵 S_b



- 矩阵转置的性质： $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$



- 定义类内散度 (within-class scatter) 矩阵 S_w :

$$S_w = \Sigma_0 + \Sigma_1 = \sum_{x \in X_0} (x - \mu_0)(x - \mu_0)^T + \sum_{x \in X_1} (x - \mu_1)(x - \mu_1)^T$$

- 定义类间散度 (between-class scatter) 矩阵 S_b :

$$S_b = (\mu_0 - \mu_1)(\mu_0 - \mu_1)^T$$

- 目标函数可以重写为一种称作广义瑞利商 (generalized Rayleigh quotient) 的形式:

$$J = \frac{w^T (\mu_0 - \mu_1)(\mu_0 - \mu_1)^T w}{w^T (\Sigma_0 + \Sigma_1) w} = \frac{w^T S_b w}{w^T S_w w}$$

3 线性判别分析的优化算法

- 广义瑞利商分子分母都是关于 $\mathbf{w}$ 的二次项，因此最优参数 $\mathbf{w}^*$ 的长度不重要，只要确定其方向，因此令 $\mathbf{w}^T S_w \mathbf{w} = 1$ ，优化目标等价为：

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} (-\mathbf{w}^T S_b \mathbf{w})$$

$$s.t. \quad \mathbf{w}^T S_w \mathbf{w} = 1$$

- 构造Lagrange函数：

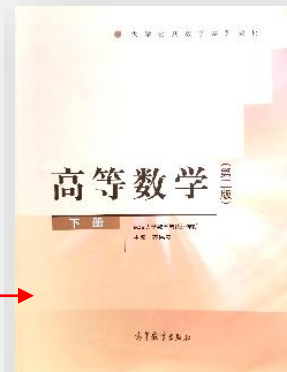
$$L(\mathbf{w}, \lambda) = \mathbf{w}^T S_b \mathbf{w} - \lambda(\mathbf{w}^T S_w \mathbf{w} - 1)$$

$$\frac{\partial L}{\partial \mathbf{w}} = S_b \mathbf{w} - \lambda S_w \mathbf{w} = 0$$

$$\text{极值解 } \mathbf{w}^* \text{ 应满足 } \Rightarrow S_w^{-1} S_b \mathbf{w}^* = \lambda \mathbf{w}^*$$

求解有约束的极值：

Lagrange乘数法



$S_w^{-1} S_b$ 矩阵的秩?

求目标函数极值问题变成了求矩阵特征值问题，最优投影方向就是这个 $S_w^{-1} S_b$ 矩阵的特征向量



- 可知 $\mathbf{w}^*$ 是 $S_w^{-1} S_b$ 的特征向量

$$\lambda \mathbf{w}^* = S_w^{-1} (\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1) (\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T \mathbf{w}^*$$

因为 $(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T \mathbf{w}^*$ 是标量, 不影响 $\mathbf{w}^*$ 的方向, 可以省去特征值分解的过程

即 $\mathbf{w}^*$ 与 $S_w^{-1} (\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)$ 共线:

$$\mathbf{w}^* = S_w^{-1} (\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)$$

- 最优投影直线的方向向量: $\mathbf{w}^* = S_w^{-1} (\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1) = (\Sigma_0 + \Sigma_1)^{-1} (\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)$

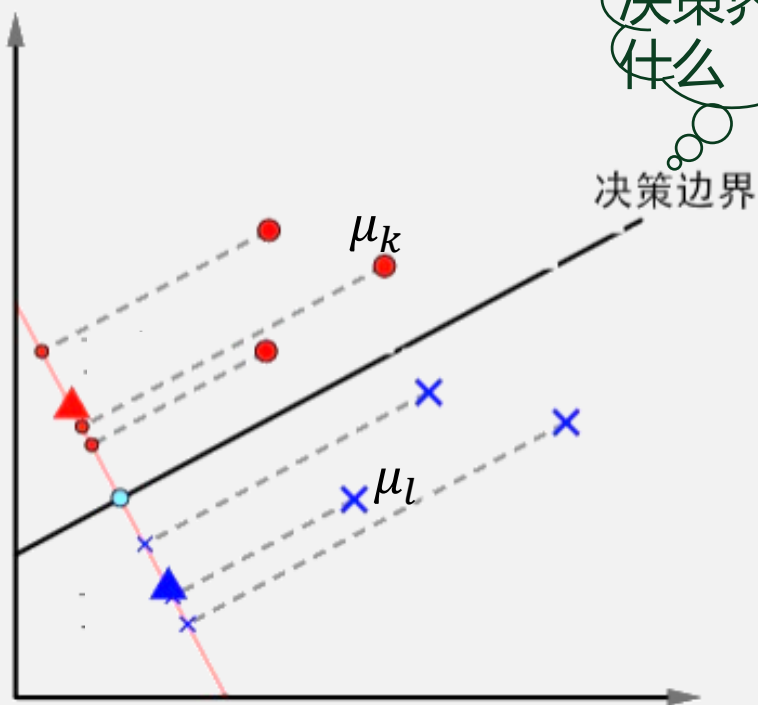
是一个与样本均值和类内协方差有关的量。实践中 [扩展阅读: LDA为什么长这样?](#)

可以通过奇异值分解进行计算。

探讨: LDA分类器是属于判别式方法, 还是生成式方法?

4 用LDA处理二分类问题

- 对 k, l 二分类问题：在投影直线确定后所有样本将会映射到一维空间中。过两类样本中心向量投影点的中点，垂直于投影直线的垂线就是分类决策界



决策界的法向量是什么

- 投影直线 $w = (\Sigma_l + \Sigma_k)^{-1}(\mu_k - \mu_l)$
- 若两类样本协方差相同: $w = \frac{1}{2} \Sigma^{-1} (\mu_k - \mu_l)$
- 投影连线的中点:

$$\frac{1}{2}(\mu_k + \mu_l)^T w = \frac{1}{4}(\mu_k + \mu_l)^T \Sigma^{-1} (\mu_k - \mu_l)$$
- 线性判别分析的**决策界**:

$$g(x) = \frac{1}{2} x^T \Sigma^{-1} (\mu_k - \mu_l) - \frac{1}{4} (\mu_k + \mu_l)^T \Sigma^{-1} (\mu_k - \mu_l) = 0$$

5 用LDA处理降维问题*

- LDA也常被视为一种经典的**监督降维**技术，应对样本的“维度爆炸”，经常被用在更为复杂的ML模型的**预处理**阶段。二分类LDA是将样本投影到一根直线上， k 类LDA至多投影到 $k-1$ 维上
- 定义全局散度（global scatter）矩阵 S_t :

$$S_t = \sum_{x \in D} (x - \bar{\mu})(x - \bar{\mu})^T$$

$\bar{\mu}$:全体样本的均值

- 定义类内散度（within-class scatter）矩阵 S_w :

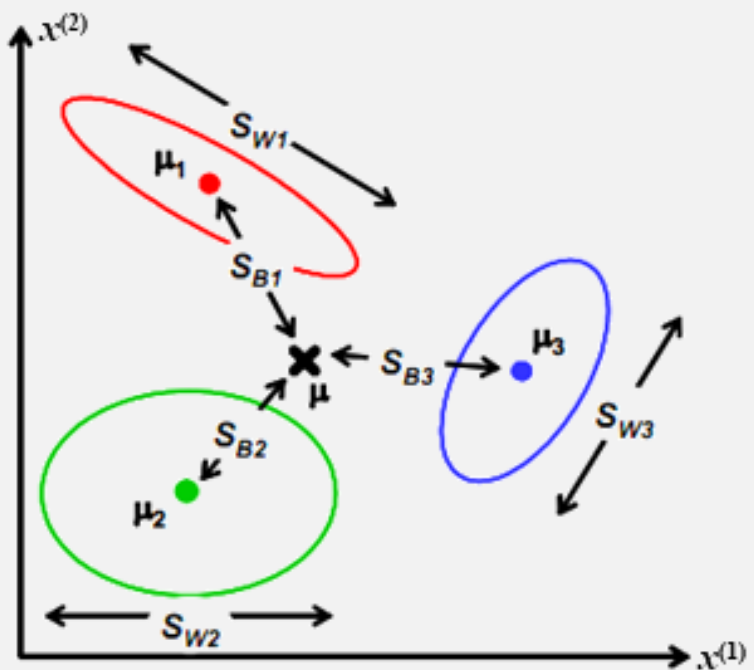
$$S_w = \sum_{i=1}^k \sum_{x \in X_i} (x - \mu_i)(x - \mu_i)^T$$

k 个类的类内散度之和

➤ 定义类间散度 (between-class scatter) 矩阵 S_b :

$$S_B = S_t - S_w = \sum_{i=1}^k |X_i| (\mu_i - \bar{\mu})(\mu_i - \bar{\mu})^T \quad |X_i|: \text{第} i \text{类样本数}$$

LDA有监督降维



输入：数据集 $D = \{x^{(1)}, x^{(1)}, \dots, x^{(m)}\}$

输出：降维后的样本集 D' ;

- 计算类内散度矩阵 S_w 和类间散度矩阵 S_B ;
- 计算矩阵 $S_w^{-1} S_B$ 的特征值;
- 找出矩阵 $S_w^{-1} S_B$ 最大的 $k-1$ 个特征值和其对应的 $k-1$ 个特征向量 $(w_1, w_2, \dots, w_{k-1})$;
- 将原始样本集投影到以 $(w_1, w_2, \dots, w_{k-1})$ 为基向量生成的低维空间中($k-1$ 维), 得到新样本集 D'

→ 第七讲



- 回顾LDA求解过程的这个步骤： $\Rightarrow S_W^{-1}S_B\mathbf{w}^* = \lambda\mathbf{w}^*$

这里 $\mathbf{w}^*$ 是 $S_W^{-1}S_B$ 的特征向量。因为两个矩阵乘积的不能超过各自的秩，所以

$$\text{rank}(S_W^{-1}S_B) \leq \min\{r(S_W^{-1}), r(S_B)\}$$

- 而类间散度矩阵 $\text{rank}(S_B) = \text{rank}\left(\sum_{i=1}^k |X_i|(\boldsymbol{\mu}_i - \bar{\boldsymbol{\mu}})(\boldsymbol{\mu}_i - \bar{\boldsymbol{\mu}})^T\right)$

$$\begin{aligned} &\leq \sum_{i=1}^k \text{rank}(|X_i|(\boldsymbol{\mu}_i - \bar{\boldsymbol{\mu}})(\boldsymbol{\mu}_i - \bar{\boldsymbol{\mu}})^T) \\ &= \sum_{i=1}^k \text{rank}(\boldsymbol{\mu}_i - \bar{\boldsymbol{\mu}}) = k \quad (\text{rank}(\boldsymbol{\mu}_i - \bar{\boldsymbol{\mu}}) = 1) \end{aligned}$$

矩阵和的秩不大于矩阵秩的和



- 矩阵的秩： $\text{rank}(A+B) \leq \text{rank}(A) + \text{rank}(B)$
 $\text{rank}(AB) \leq \min\{\text{rank}(A), \text{rank}(B)\}$



μ_i 是每个类 X_i 的均值向量，如果求出前 $k-1$ 个 μ_i ，最后一个 μ_k 可以由前 $k-1$ 个 μ_i 线性表出，因此 S_B 最多有 $k-1$ 个特征向量，秩最大为 $k-1$

$$\text{rank}(S_w^{-1}S_B) \leq k - 1$$

- 所以二分类LDA是将样本投影到一根直线上， k 分类LDA至多投影到 $k-1$ 维上。作为降维工具，LDA降维是一种有监督降维（需要知道**类标记**），且将 **d 维样本最多降至 $k-1$ 维**



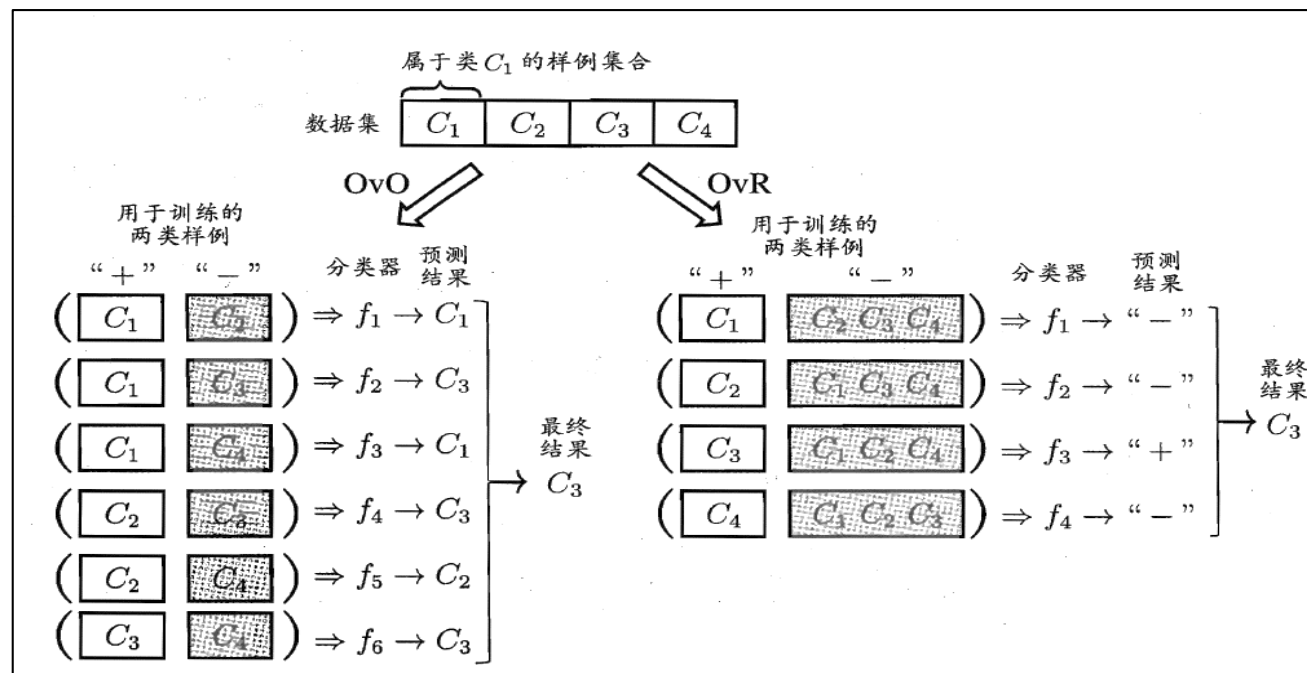
1 用多次二分类实现多分类

- 现实中我们常遇到多分类学习任务。通常采用将一个多分类任务**拆分**为若干个二分类任务，将二分类学习推广到多分类学习。在测试时，对这些分类器的预测结果进行**集成**以获得最终结果

经典拆分策略

- 一对一, One vs. One, OvO
- 一对其余, One vs. Rest, OvR
- 多对多, Many vs. Many, MvM *

- OvO将 N 个类两两配对，构造 $k(k-1)/2$ 个分类器，把它们预测最多的类别作为最终的分类结果（投票）
- OvR则是每次将一个类作为正例、其他类的样例作为反例来训练 N 个分类器。在测试时若仅有一个分类器预测为正类，则对应的类别标记作为最终分类结果



MvM拆分策略
请参阅
西瓜书p64-
65纠错输出
码

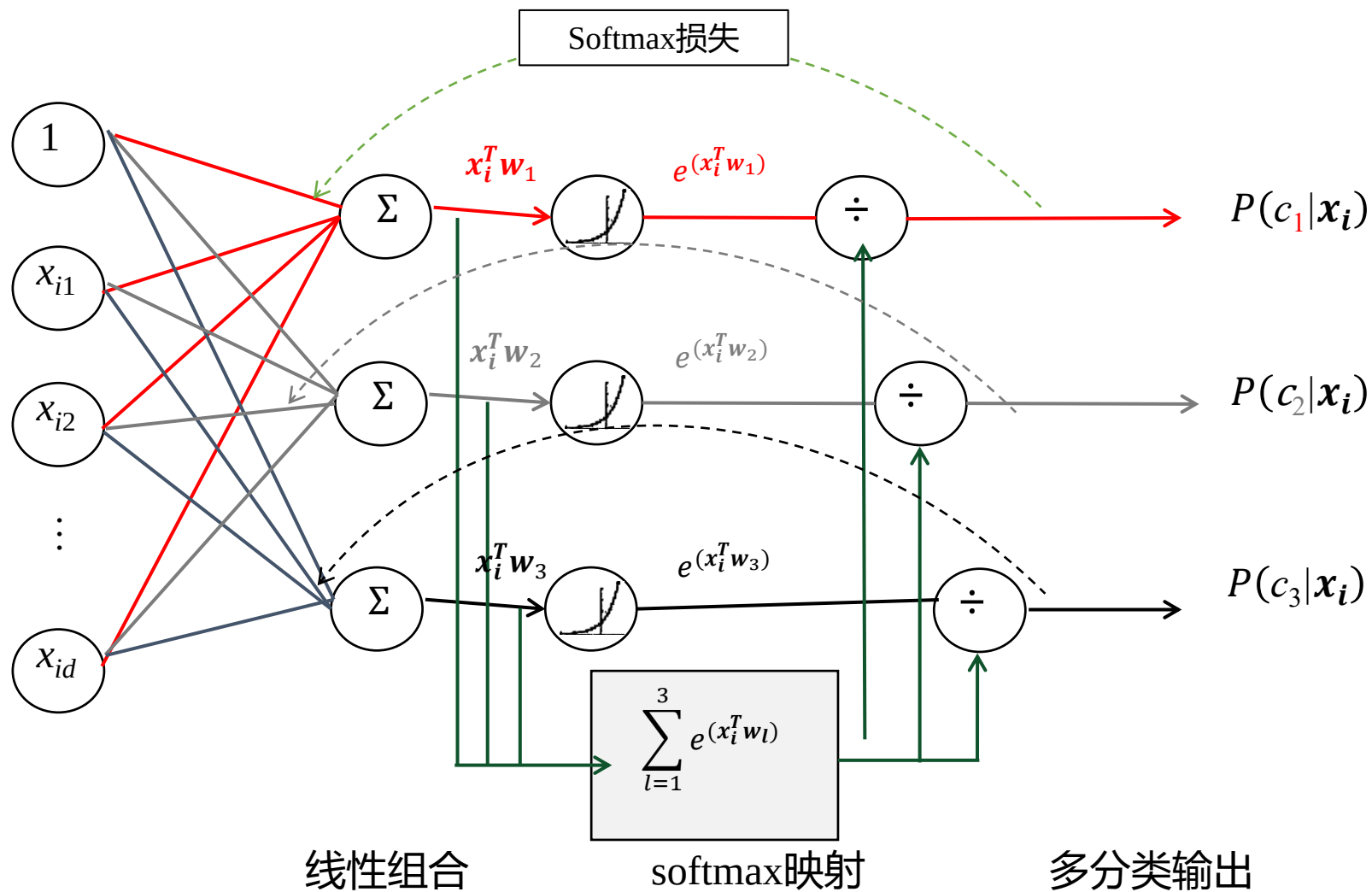
2 引入Softmax函数

- 对数几率回归使用的**Logistic函数**是一个典型的Sigmoid函数（S型函数），能够将样本的线性回归预测值映射到(0,1)之间，代表其属于正类的概率，因此适用于二分类问题
- 对于 k 分类系统，引入名为“**软最大 Softmax 函数**”：考虑样本 $D = \{(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots, (\mathbf{x}_m, \mathbf{y}_m)\}$ ，标记 $\mathbf{y}_i$ 是形如 $(0, 0, 0, 1, 0, 0, \dots, 0)^T$ 的 $k \times 1$ 向量。输出 $\hat{\mathbf{y}}_i$ 为 $(\hat{c}_1, \hat{c}_2, \hat{c}_3, \dots, \hat{c}_k)^T$ 的 $k \times 1$ 向量，其中 $\hat{c}_j$ 为样本 $\mathbf{x}_i$ 属于 c_j 类的概率，满足：

$$\hat{c}_j = P(c_j | \mathbf{x}_i) = \frac{e^{(\mathbf{x}_i^T \mathbf{w}_j)}}{\sum_{l=1}^k e^{(\mathbf{x}_i^T \mathbf{w}_l)}}$$

其中 $\mathbf{w}_l$ 是第 l 类（第 l 个广义线性回归）使用的参数向量 $[w_{l1}, w_{l1}, \dots, w_{ld}, w_{l0}]$

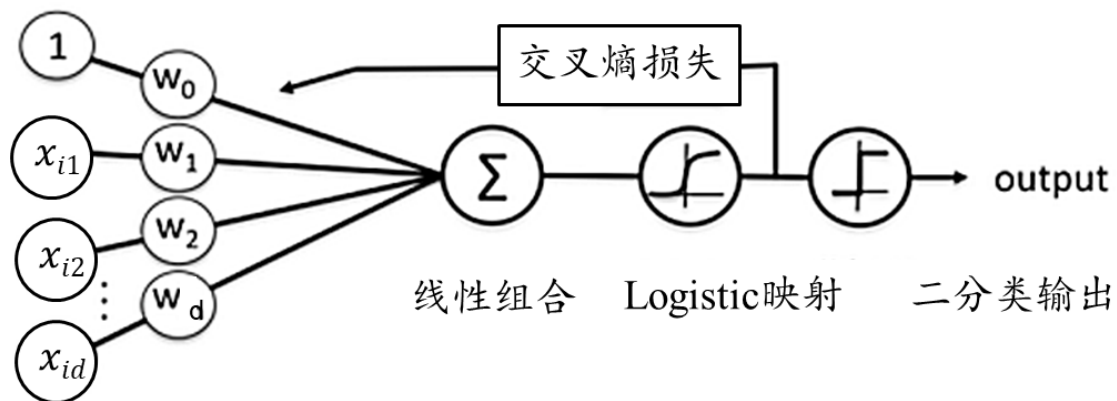
增广偏置参数



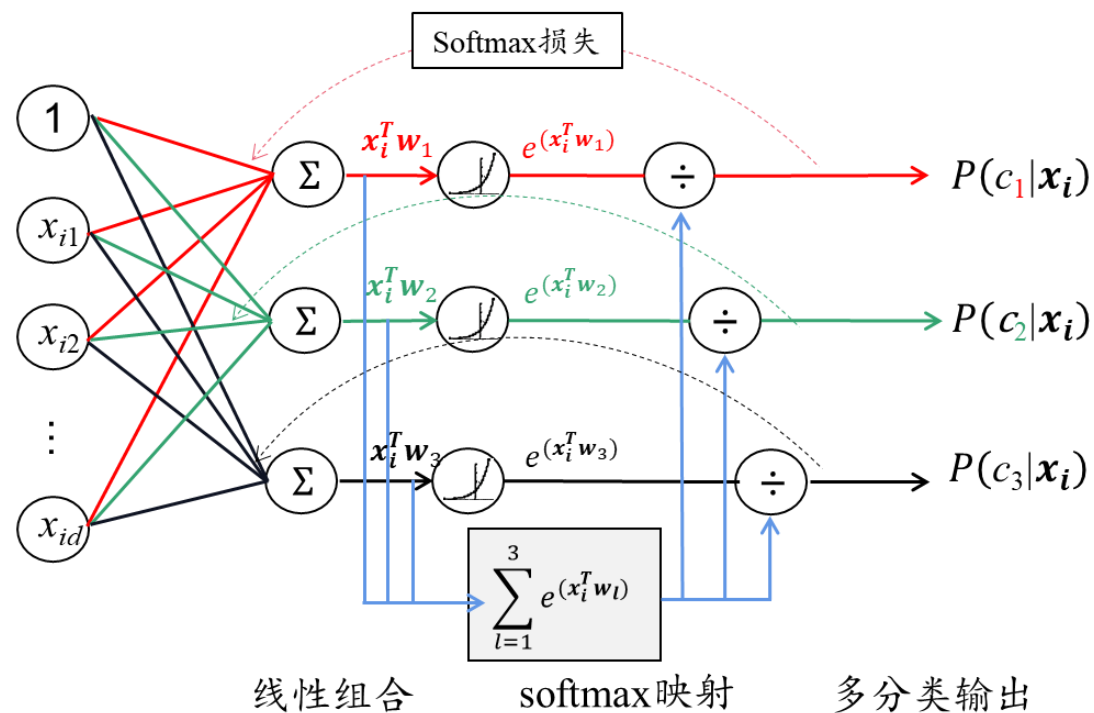
Softmax函数的计算流程

➤ Sigmoid函数与Softmax函数对比

Logistic



Softmax





例子. 在手写数字识别任务中, 如果样本是数字“5”, 那么真实分布应该为: $[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]$ 。若输出的分布为: $[0.1, 0.1, 0, 0, 0, 0.7, 0, 0.1, 0, 0]$, 则 $k=10$, 该样本的10分类交叉熵损失为:

10分类, 写为10项和

$$Loss = -0 \times \log 0.1 - 0 \times \log 0.1 - 0 \times \log 0 - 0 \times \log 0 - 0 \times \log 0 - 1 \times \log 0.7 - 0 \times \log 0 - 0 \times \log 0.1 - 0 \times \log 0 - 0 \times \log 0 \approx 0.3567$$

$0 \log 0 = 0$

如果输出的分布为: $[0.2, 0.3, 0.1, 0, 0, 0.3, 0.1, 0, 0, 0]$, 那么损失函数得:

$$Loss = -0 \times \log 0.2 - 0 \times \log 0.3 - 0 \times \log 0.1 - 0 \times \log 0 - 0 \times \log 0 - 1 \times \log 0.3 - 0 \times \log 0.1 - 0 \times \log 0 - 0 \times \log 0 - 0 \times \log 0 \approx 1.2040$$

上述两种情况对比, 第一个分布的损失明显低于第二个分布的损失, 说明第一个分布更接近于真实分布

注: 同样使用 $\ln$ 计算

- 线性关系简单、易于理解和计算，而非线性关系更为复杂和多样化
- **线性思维**是一种直线的、单向的、缺乏变化的思维方式。它倾向于将事物简化为一连串的因果关系，按照时间顺序或逻辑顺序逐步推导。非线性思维不拘泥于因果关系、分类和区分，注重整体性、相对性、动态性、创造性和和谐性

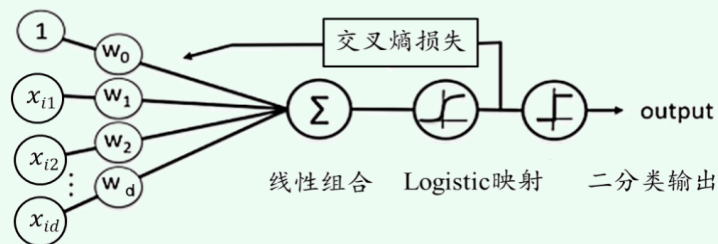
- [1]. 统计学-线性回归中的假设检验[EB/OL][2018-01-08]. <https://zhuanlan.zhihu.com/p/32750128>
- [2]. Jackeeeee M.一文读懂信息熵、交叉熵和相对熵（KL散度）[EB/OL][2022-12-21]. <https://zhuanlan.zhihu.com/p/593362066>
- [3]. DeAlVe.教科书上的LDA为什么长这样？ [EB/OL][2018-08-30]. https://www.sohu.com/a/250923766_500659

本章小结

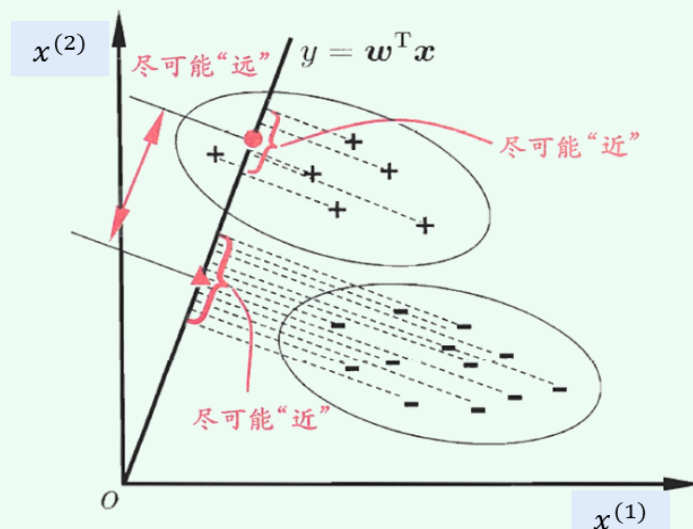


武汉大学
WUHAN UNIVERSITY

- 线性模型是很多机器学习模型的基础，可以用于处理分类、回归以及降维问题。线性模型可以通过联系函数、基函数以及核函数方法，实现**非线性推广**，构成非线性分类器或回归器
- 对数几率回归LR、线性判别分析LDA的模型设计思想、学习策略与优化方法需认真体会、对比以及动手实验
- **课后请阅读西瓜书第三章**



Logistic回归 (LR)



线性判别分析 (LDA)



武汉大学
WUHAN UNIVERSITY

End of This Part
